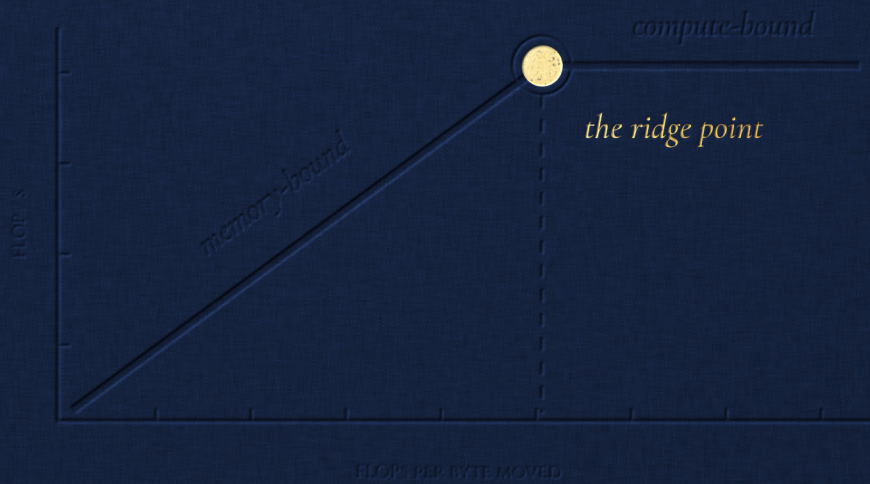


VOL. 01

RIDGEPOINT

*a single-GPU LLM inference engine,
built and measured from the scheduler down*

LAB NOTEBOOK



RTX 4050 LAPTOP GPU · 6 GB · QWEN2.5-1.5B · BF16

BENCHMARKED AGAINST HF GENERATE · VLLM · LLAMA.CPP

NAME

STARTED

NO.

DESIGN · FUNDAMENTALS · EXPERIMENTS · 30-DAY PLAN

How to read this notebook

The full design of Ridgepoint as working notes: the engine, the physics that constrains it on a 6 GB laptop GPU, the versions you build, the experiments that prove each one, and the plan to finish in 30 days.

No benchmark number in this notebook is a result – every result is yours to measure. Blank tables and [X] placeholders are deliberate.

The ink code

blue pen

Facts, definitions and the design itself.

red pen

Warnings, mistakes people actually make, and things you must not skip.

pencil

Estimates from spec sheets. Every one of them gets replaced by a measurement.

highlighter

The one phrase on the page to remember.

ESTIMATE

Arithmetic, not a benchmark. Your report compares measurements against these.

The boxes

*dashed box =
formula*

Derive it yourself from
config.json and
measured hardware
numbers.

sticky = remember

The insight that makes
the page make sense.

red box = don't

A classic mistake, and
what it costs.

The one rule behind every page

Every optimization answers two questions: what bottleneck are we fixing, and what did it cost us? A profile or measurement isolates the bottleneck; you predict the gain from first principles; you measure it; you write down the cost. Keep the failed hypotheses.

Contents

Before you start

0	Hardware facts that shape the design	4
---	--	---

The project

1	Project name	5
2	One-sentence description	5
3	Engineering problem	6
4	Why this project is high-signal	7
5	Architecture	9
6	Request lifecycle	14

The physics

7	Inference fundamentals you need first	17
---	---	----

The build

8	Baseline system (V0)	25
9	Optimization roadmap (V1-V7)	27
10	Repository structure	37

Measurement

11	Benchmarking framework	38
12	Experiments E1-E17	41
13	Failure-mode testing	45
14	Profiling strategy	49
15	Tests	52
16	Dashboard	54

Presenting it

17	README structure	55
18	Resume bullets	56
19	LinkedIn project description	56
20	Interview questions about this project	57
21	What you must be able to answer first	59
22	Anti-patterns	60
23	Production evolution: 1 → 8 → 100 GPUs	61

Doing it

24	30-day build plan	63
25	Final recruiter test	66

Appendices

A	Formula cheat sheet	68
B	Worked numbers from config.json	70
C	Glossary	72
D	Experiment log pages	74

0 Hardware facts that shape the design

Three facts drive most of the decisions in this notebook. Learn them before you write a line of engine code.

Fact 1 — 6 GB will not hold a 3B model in 16-bit

Qwen2.5-3B's weights alone are about 6.2 GB in BF16. The primary model therefore has to be around 1.5B parameters. Qwen2.5-1.5B-Instruct is about 3.1 GB, which leaves roughly 1.5–2 GB for the KV cache.

- 3–4B models enter only through weight quantization (V7).
- 8B models enter only as 4-bit with partial CPU offload (E14).
- This helps the project. A 1.5 GB KV pool makes memory management a real, visible problem instead of a theoretical one.

Fact 2 — memory bandwidth is the scarce resource

The RTX 4050 Laptop GPU has 20 SMs and a 96-bit GDDR6 bus. At spec memory clocks that is about 192 GB/s. Verify the numbers for your laptop.

- The laptop maker sets the power limit, somewhere between roughly 35 W and 115 W. Two "RTX 4050" laptops can differ a lot.
- You measure the real numbers first (E1). Everything afterwards is interpreted against them.

Fact 3 — laptop thermals are your biggest source of noise

- ☐ Run plugged in, with a fixed fan profile.
- ☐ Log clocks and temperature in every run, and discard runs that throttle.
- ☐ Use native Linux, not WSL2. WSL2 limits GPU profiling and adds variance.
- ☐ Have the integrated GPU drive the display, so nothing else occupies your 6 GB.

The model plan

Use one model family throughout, so the tokenizer and architecture stay constant across every experiment.

Role	Model	Why
Primary	Qwen2.5-1.5B-Instruct, BF16	Fits with a usable KV pool
CPU-overhead contrast	Qwen2.5-0.5B	Short GPU steps expose host overhead
KV-heavy contrast	Qwen3-1.7B	Similar weight bytes, about 4× the KV bytes per token
Quantization	Qwen2.5-3B at INT8 / INT4	Only fits once quantized
Offload experiment	Llama-3.1-8B, 4-bit GGUF, in llama.cpp	Two-tier bandwidth model

Qwen3 vs Qwen2.5 — the two small differences

Qwen3 drops the QKV bias and adds a per-head RMSNorm on Q and K. If newer small dense GQA models exist when you start, substitute one — then keep the choice fixed for the whole project.

1 Project name

RIDGEPOINT

a single-GPU LLM inference engine

The ridge point is where a roofline changes from memory-bound to compute-bound. The project is about three things:

1. Moving decode toward the ridge point — batching.
2. Removing the bytes that keep you away from it — paging, prefix caching, quantization.
3. Always knowing where you are relative to it — the performance model.

Inference engineers will recognize the reference, and it tells a reviewer in one word what the repo is about.

2 One-sentence description

Ridgepoint is a single-GPU LLM inference engine written from the scheduler down in PyTorch — paged KV cache, continuous batching with chunked prefill, prefix caching, and memory- and SLO-aware admission control — built as a sequence of measured versions against a naive baseline and vLLM on replayable workloads, with

a first-principles performance model that predicts step latency from measured memory bandwidth.

► Short form (for the resume header)

A from-scratch LLM serving engine (paged KV, continuous batching, prefix caching, SLO-aware scheduling) benchmarked version-by-version against vLLM on a 6 GB GPU.

3 Engineering problem

The setup. You have one GPU that is short on both memory and bandwidth. Requests arrive at random times, with varied prompt and output lengths.

the goal

Maximize goodput: the highest sustained request rate at which P95 TTFT and P95 TPOT stay within their SLOs — while:

- it never crashes or leaks memory, including under client disconnects, overload and malformed input;
- under overload it rejects requests early with a retry hint, instead of letting every request time out;
- every engine step's latency can be explained from a measured model of the hardware.

Five constraints that fight each other

1. Decode is bandwidth-bound. Producing one token per sequence requires streaming every weight byte. Only batching amortizes that cost.
2. Batching consumes KV memory. KV grows linearly with every sequence's context, and it comes out of a pool of about 1.5–2 GB.
3. Prefill is long and compute-bound. Running a prefill stalls every in-flight stream; deferring it inflates TTFT.
4. Arrivals are random. Near saturation, queue wait grows nonlinearly.
5. The Python host must keep up. It has to build inputs and launch several hundred kernels every 15–30 ms without leaving the GPU idle.

Every version in §9 relieves one of these five and makes another one worse. That tension is the story.

SLOs – freeze them early

chat workload SLOs

$$P95 \text{ TTFT} \leq 1.0 \text{ s} \quad \cdot \quad P95 \text{ TPOT} \leq 75 \text{ ms}$$

- By the FLOP estimate in §7.2, an unloaded 2k-token prefill costs about 0.3–0.4 s on this GPU. A 1.0 s TTFT target leaves room for queueing.
- 75 ms per token is about 13 tokens/s – faster than people read.

the SLO rule

Measure unloaded latency in V0. If the floor is higher than estimated, adjust the SLOs once, write down why, and then never change them. Report how results change with SLO tightness in E5.

4 Why this project is high-signal

► It works in the layer inference teams actually build

Anyone can launch vLLM. Very few new graduates can explain what vLLM's scheduler decides every step, and why. Here you write that decision loop, the memory manager beneath it, and the runtime it drives.

► The story is causal

Each version follows the same loop – the daily work of a performance engineer:

profile isolates a bottleneck → predict the gain from first principles →
measure it → state what it cost

► It is anchored to external references

vLLM and llama.cpp run the same traces through the same harness. "My numbers went up" becomes "my engine reaches X% of vLLM's goodput on this workload, and the remaining gap comes from here." **Without that, every number in the repo only compares you to yourself.**

► Correctness is proven, not assumed

- ✓ logit equivalence against Hugging Face;
- ✓ identical outputs for cache hit vs. miss;
- ✓ identical outputs for chunked vs. whole prefill;
- ✓ identical outputs for preempted vs. uninterrupted sequences.

A fast engine that produces wrong output is the most common failure in this space.

► *Stress behavior is designed in*

Invariants such as "zero leaked KV blocks after 10,000 random disconnects", measured overload behavior, and engine-crash recovery.

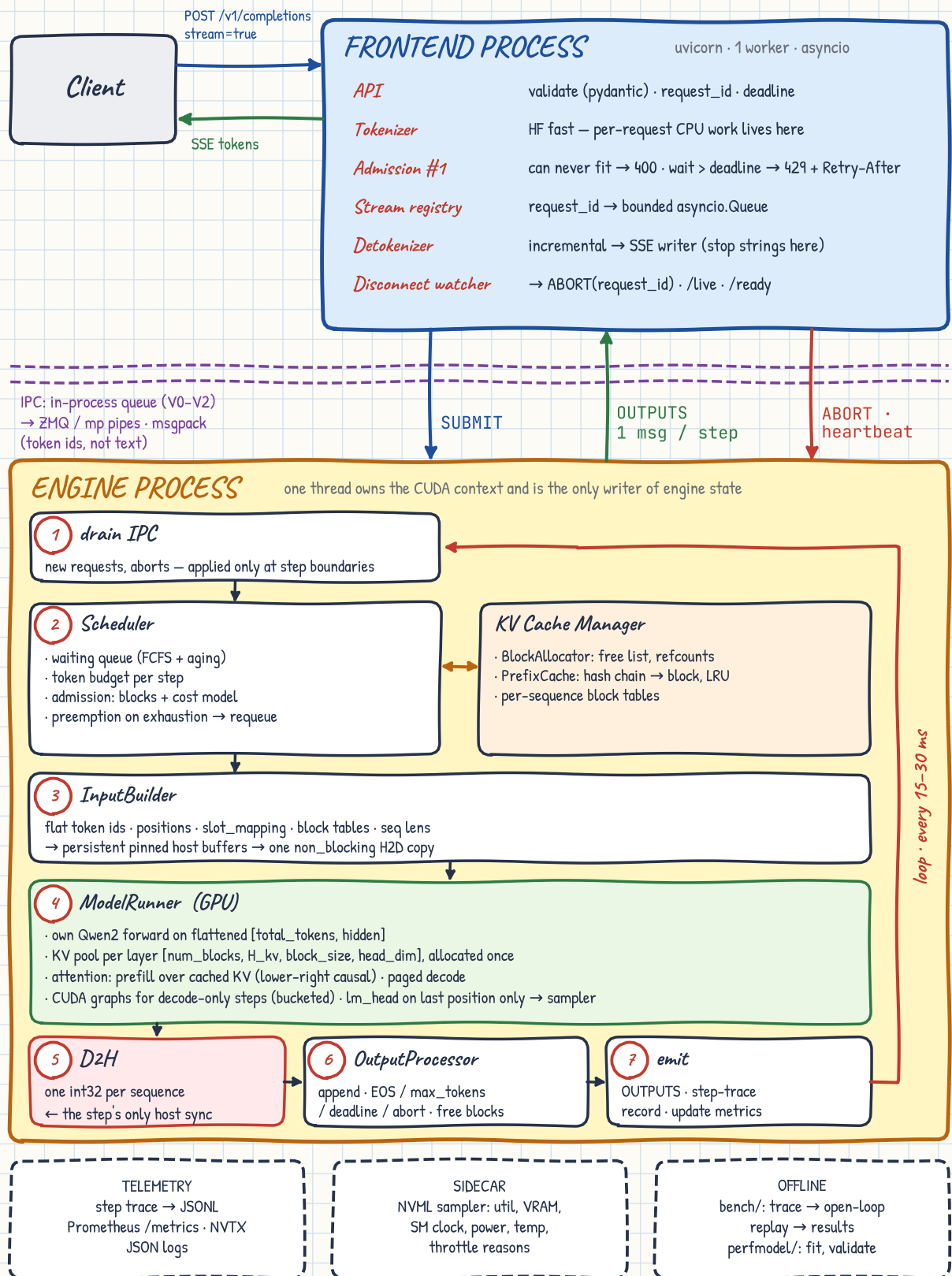
► *It predicts, not just measures*

A validated step-time model shows you understand the hardware well enough to forecast behavior.

what it does NOT prove — say so in the README

Multi-GPU experience, CUTLASS-level kernel skill, or production operations. Overclaiming is what makes interviewers dig.

5 Architecture



two processes, one GPU owner, one decision loop — sketch of the full system

The whole engine is one function, called in a loop

You should be able to write this on a whiteboard from memory and explain every line.

src/ridgepoint/engine/loop.py

```
def step(self):
    self.drain_ipc()                # new requests + aborts, applied only here
    batch = self.scheduler.schedule() # decodes first, then prefill chunks in budget
    if batch.empty():
        return self.wait_for_work()
    inputs = self.input_builder.build(batch) # fill pinned buffers, one H2D copy
    next_ids = self.runner.execute(inputs)   # graph replay or eager; device [num_seqs]
    ids = next_ids.tolist()                # the one host↔device sync per step
    done = self.output_processor.apply(batch, ids) # stop checks, free blocks, cache insert
    self.ipc.send_outputs(batch, ids, done)
    self.trace.record(batch, timings)
```

why this shape

Only this thread mutates scheduler and KV state, so races are impossible by construction. Abort and new requests are applied only at the top of a step, so abort latency is at most one step.

Components

► Frontend

- Owns HTTP, validation, tokenization, incremental detokenization, SSE and disconnect detection. It holds no GPU state.
- All per-request CPU work lives here, so it cannot delay an engine step.
- The static admission gate rejects two kinds of request:
 - can never succeed: prompt longer than `max_model_len`, or prompt plus `max_tokens` larger than the whole KV pool;
 - cannot succeed in time: estimated queue wait greater than the deadline. The estimate uses queued tokens divided by the engine's published throughput.
- Run exactly one uvicorn worker. Multiple workers means multiple engines competing for one GPU.

► IPC

- Messages carry token ids, not text.
- The engine sends one OUTPUTS message per step containing every sequence's new tokens, so the IPC is batched too.
- Heartbeats let the frontend detect a dead engine.

► Scheduler

- Makes one decision per step: which sequences run; how many prompt tokens of each waiting sequence to prefill; whom to preempt.
- It is pure Python and **deterministic given its state and a clock**. That means it is unit-testable on CPU with a fake executor.

► KV Cache Manager

- Owns the physical blocks.
- BlockAllocator: a free list plus reference counts.
- PrefixCache: maps `hash(parent_hash, block_token_ids)` to a physical block. Blocks whose refcount reaches zero sit in an LRU evictable set until they are reclaimed.
- Debug-mode invariant: `free + in_use + evictable == total`, and every block is in exactly one set.

► InputBuilder

Turns the scheduled batch into GPU tensors, and **allocates nothing per step**:

- flattened token ids;
- positions (for RoPE);
- `slot_mapping` — where each new token's K and V are written;
- padded block tables;
- context lengths;
- query start offsets;
- the index of each sequence's last token.

► ModelRunner

Your own Qwen2 forward pass:

the forward pass, one layer at a time

```
embedding
for each of 28 layers:
    RMSNorm → QKV (with bias) → RoPE → write K,V into the pool → attention
    → o_proj → residual → RMSNorm → gate/up → SiLU·mul → down → residual
final norm → lm_head on the selected rows only → sampler
```

- Dense ops run on the whole flattened batch. Every non-attention op is per-token, so a batch is just a concatenation. **That is why continuous batching needs no padding.**
- Attention is the only op that sees sequence structure.
- Attention backends are pluggable:
 - a reference path (gather + SDPA) that serves as the correctness oracle;
 - a Triton paged decode kernel (stretch goal);
 - FlashInfer, as an external comparison.

► OutputProcessor

- Applies stop conditions: EOS unless `ignore_eos`, `max_tokens`, stop token ids, deadline, abort.

- Releases blocks. Full blocks go to the prefix cache; a partial final block goes back to the free list.

► Telemetry

A per-step trace record — the single artifact that feeds the cost model, the timeline plots and most of your debugging:

- `step_id`, wall-clock times and CUDA-event GPU time;
- `num_decode_seqs`, `num_prefill_tokens` and Σ context;
- blocks used, free and evictable;
- preemptions, eager vs. graph execution, and batch bucket.

Key design decisions

Write each of these as a one-page ADR in `docs/adr/`.

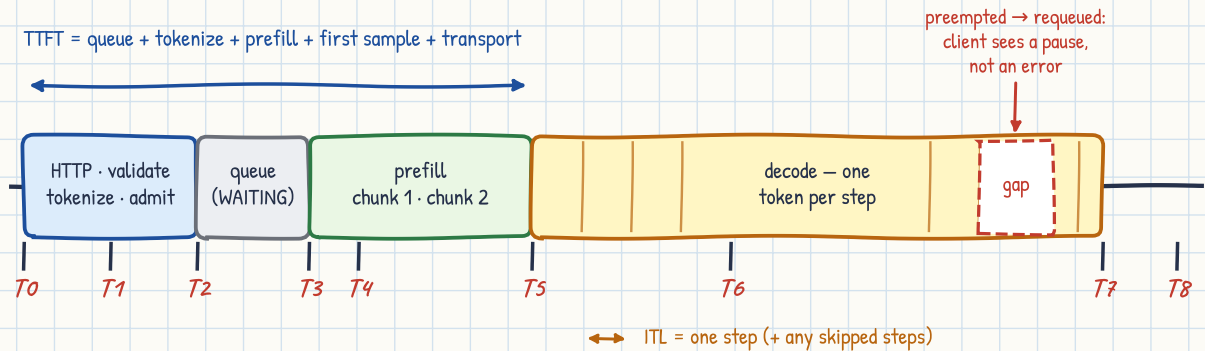
1. Single-writer engine loop. Only the engine thread mutates scheduler and KV state, so races are impossible by construction. Concurrency is handled at the IPC boundary.
2. No Redis in the hot path. The scheduler inspects the waiting queue every 15–30 ms, so the queue must be an in-process data structure. A network hop per scheduling decision (or per token) buys nothing on one machine.
3. Separate frontend and engine processes — but only once you can justify it. Start with an in-process engine thread behind an `EngineClient` interface. Split into two processes when you can show either:
 - GIL interference: tokenization, detokenization, JSON and SSE for 200 streams adding jitter to engine steps (E12b);
 - the fault-isolation argument: a sticky CUDA error such as an illegal memory access poisons the CUDA context and forces a process restart, which should not drop the frontend.
4. The KV pool is allocated once at startup. It is sized from measured free memory after a profiling forward pass at the maximum token budget. It never grows.
5. Versions are configurations of one codebase, plus git tags (`v0 ... v7`). Any version can then be re-run with today's harness. **Copy-pasted `v1/`, `v2/` folders are a red flag.**
6. The benchmark harness is engine-agnostic. It speaks OpenAI-compatible SSE, so the same trace runs against the HF baseline, every Ridgepoint version, vLLM and llama.cpp.

What you build vs. what you use

You implement	You use (and must be able to explain)
Engine loop, sequence state machine, scheduler, admission, preemption	HF tokenizers, safetensors
Block allocator, block tables, prefix cache	cuBLAS GEMMs via PyTorch; SDPA / flash for dense prefill attention
Model forward pass, RoPE, GQA plumbing, KV write path	FlashInfer, as a comparison attention backend
Reference paged attention; Triton paged decode kernel (stretch)	INT4 GEMM kernels (torchao / Marlin-class) in V7
Input builder, CUDA graph capture and replay, sampler	FastAPI, uvicorn, prometheus_client, pynvml
Streaming, abort path, incremental detokenizer	vLLM, llama.cpp — only as external baselines
Benchmark harness, performance model, failure injection	Prometheus and Grafana (optional)

6 Request lifecycle

From HTTP POST to the last SSE event. Every T-mark is a timestamp you log.



T5 first token (the D2H of sampled ids is the sync) · T7 finish: blocks → prefix cache · T8 accounting & logs

one request's life — T-marks are logged timestamps

T0

Arrival

- FastAPI parses the JSON and pydantic validates it. Invalid requests get a 422.
- The request gets a `request_id` and `deadline = now + timeout`.
- Log `request.received`.

T1

Tokenized

Apply the chat template if the endpoint needs one, then encode. Run the admission checks in this order:

1. prompt longer than `max_model_len` → 400;
2. prompt plus `max_tokens` larger than the total KV pool → 400 (it can never run);
3. estimated queue wait greater than the deadline → 429 with `Retry-After`.

If the request passes, register a bounded `asyncio.Queue` for its stream and send SUBMIT to the engine.

T2

Enqueued

- At the start of its next step, the engine drains IPC.
- It creates a `Sequence` in state WAITING and adds it to the waiting queue.

T3**Scheduled**

- The scheduler first reserves budget for the running decodes: one token each, plus a new block for any sequence crossing a block boundary. If no block is free, it preempts.
- It then walks the waiting queue. For each candidate:
 - Prefix-cache lookup: hash the prompt's full blocks in order and take the longest chain of hits. Those blocks are shared by incrementing their refcounts, and `num_computed_tokens = hits × block_size`. **If the entire prompt hits, leave the last token uncached so there is a position to compute logits from.**
 - Admission: there must be enough free plus evictable blocks for the uncached part plus a watermark reserve. From V5 on, the cost model must also agree that the added prefill chunk keeps predicted step time within the TPOT budget.
 - Chunk size: `min(remaining prompt, remaining token budget)`.

T4**Prefill (one or more steps)**

The chunk's tokens are packed alongside the decode tokens. In every layer:

- Q, K and V are computed for the chunk tokens.
- K and V are scattered into the pool at `slot_mapping`.
- The chunk's queries attend to [cached prefix KV || earlier chunks || this chunk], using a **lower-right-aligned causal mask**.
- Only the final chunk's last position goes through `lm_head`.

T5**First token**

- The sampler produces a token id on the GPU.
- The D2H copy of all sampled ids is the step's sync point.
- The OutputProcessor appends the token and the engine sends OUTPUTS.
- The frontend's detokenizer may emit an empty string if the token completes only part of a UTF-8 character.

define TTFT precisely

The first token as the client observes it, computed from token counts the server includes in each SSE event — not from visible text. Use that definition everywhere.

T6**Decode**

- Each running sequence contributes one token per step.
- Its new K and V go into its last block; a new block is allocated every `block_size` tokens.
- Attention reads its entire KV through the block table.
- A sequence's ITL equals the duration of the step, plus any steps in which it was not scheduled.

pre-empt**Preemption path**

- Triggered when a running sequence needs a block and none are free.
- The victim — the most recently admitted sequence — has its blocks released. Its full blocks may stay in the prefix cache as evictable, so recompute may later hit the cache.
- It is queued at the head of the waiting queue, with prompt plus generated tokens as its new prompt.
- The client sees a gap in the stream, not an error.

T7**Finish**

- A request finishes on EOS (unless `ignore_eos`), `max_tokens`, a stop condition, deadline or abort.
- Its full blocks are inserted into the prefix cache and the partial block is freed.
- The final SSE event carries `finish_reason`, usage, and a server timing breakdown: `queue_ms`, `prefill_ms`, `decode_ms`, `cached_tokens`, `preemptions`. Then `data: [DONE]`.

T8**Accounting**

Histograms are observed, and `request.finished` is logged with every timestamp and the request id.

abort**Abort path**

- The frontend detects a disconnect in one of two ways: the streaming task is cancelled, or an explicit `is_disconnected()` poll fires. **Write a test proving which one actually fires in your stack.**
- It then sends ABORT.
- The engine applies the abort at the next step boundary, so abort latency is at most one step.
- The frontend never mutates engine state.

7 Inference fundamentals you need first

Everything later in this notebook is interpreted against the numbers on these pages. Re-derive every one from `config.json` and your E1 measurements.

7.1 Prefill vs. decode

prefill

- Every weight matrix multiplies a $[P, d]$ activation (P = prompt tokens).
- FLOPs $\approx 2 \cdot N \cdot P$ for N parameters, plus the quadratic attention term.
- Arithmetic intensity grows with P , so it is **compute-bound** once P reaches a few hundred tokens.
- Output: K and V for every prompt position, plus logits for the first new token.

decode

- Every weight matrix multiplies a skinny $[B, d]$ activation (B = sequences in the batch).
- To produce B tokens you stream all W bytes of weights, plus each sequence's KV cache.
- So it is **memory-bandwidth-bound** at every practical batch size on this card.

7.2 Roofline for your GPU

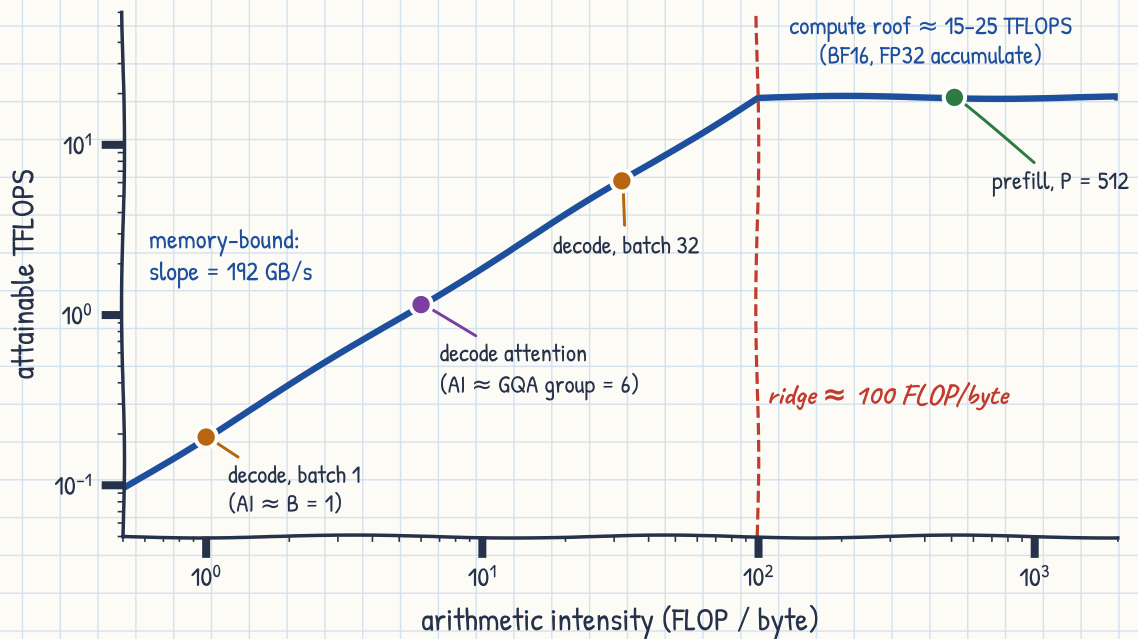
definitions

$$\text{arithmetic intensity (AI)} = \text{FLOPs} \div \text{bytes moved}$$

$$\text{ridge point} = \text{peak FLOP/s} \div \text{peak bandwidth}$$

Spec-level estimates **ESTIMATE** — you replace these with E1 measurements:

- Bandwidth $\approx 192 \text{ GB/s}$.
- Tensor-core BF16/FP16 throughput with FP32 accumulation (the mode PyTorch uses) is on the order of 15–25 TFLOPS depending on clocks. GeForce Ada runs FP32 accumulation at half the FP16-accumulation rate.
- Ridge point $\approx 100 \text{ FLOP/byte}$.



roofline from spec numbers — redraw it with your E1 measurements

► What that means for decode

- Weights: a 2-byte weight is used for one multiply-add (2 FLOPs) per sequence, so $AI \approx B$. The weights only reach the ridge point around $B \approx 100$.
- Attention: each K/V element is read once and used by every query head in its GQA group, so $AI \approx$ group size. That is 6 for Qwen2.5-1.5B (12 query heads $\div$ 2 KV heads), and batching does not raise it because each sequence's KV is private.
- Conclusion: almost all practical decode on this card is memory-bound.

► Derived bounds — arithmetic, not benchmarks

batch-1 decode ceiling (Qwen2.5-1.5B)

$$3.1 \text{ GB} \div 192 \text{ GB/s} \approx 16 \text{ ms / token} \approx 62 \text{ tok/s}$$

measured TPOT $\div$ this ceiling = your first efficiency number

decode step time model

$$t_{\text{step}} \approx \alpha + (w + \sum_i \text{KV_bytes}(\text{ctx}_i)) \div \text{BW}$$

α = fixed overhead (launches, host work, sampler); fitted in E15

a full-pool decode step reads roughly all the VRAM you are using

$$(3.1 + 1.8) \text{ GB} \div 192 \text{ GB/s} \approx 25 \text{ ms}$$

holds regardless of the model's KV geometry — when the pool is full, $\Sigma \text{ KV bytes} \approx \text{pool size}$

so why does GQA matter?

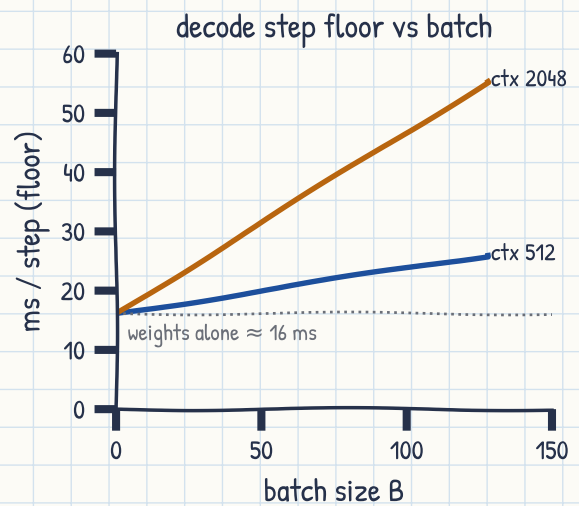
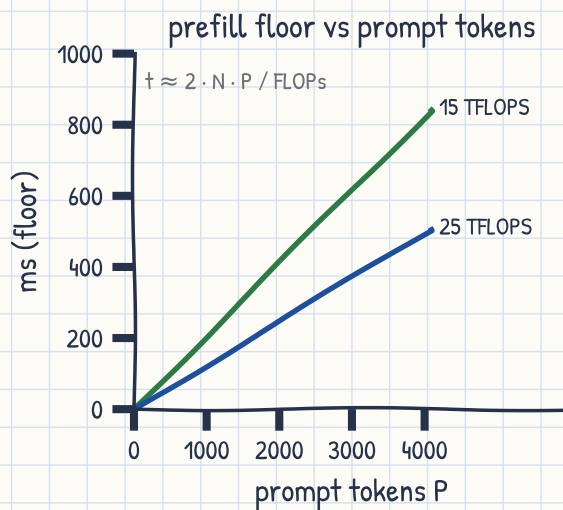
KV geometry changes how many sequences fit into that fixed step time. At 28 KiB/token vs. 112 KiB/token, you get 4x the concurrent tokens for the same step time. That is why GQA and KV quantization are throughput optimizations, not just capacity optimizations.

prefill floor

$$t_{\text{prefill}} \geq 2 \times 1.54 \times 10^9 \times P \div \text{achieved FLOP/s}$$

at $\approx 15 \text{ TFLOPS}$: 1k prompt tokens $\approx 0.2 \text{ s}$ · 2k tokens $\approx 0.4 \text{ s}$

Two long prompts queued ahead of a request are enough to break a 1 s TTFT SLO. That is why prefill scheduling matters.



the two regimes as floors from the formulas (spec bandwidth, no α) — E2 plots the measured version

7.3 KV cache

► What is stored

- For every layer, every past position and every KV head: the K vector (after RoPE) and the V vector.
- Q is not stored; it is only needed at the step that computes it.

- Every future token attends to every past token, so KV size is linear in context length and summed across sequences.

KV bytes per token

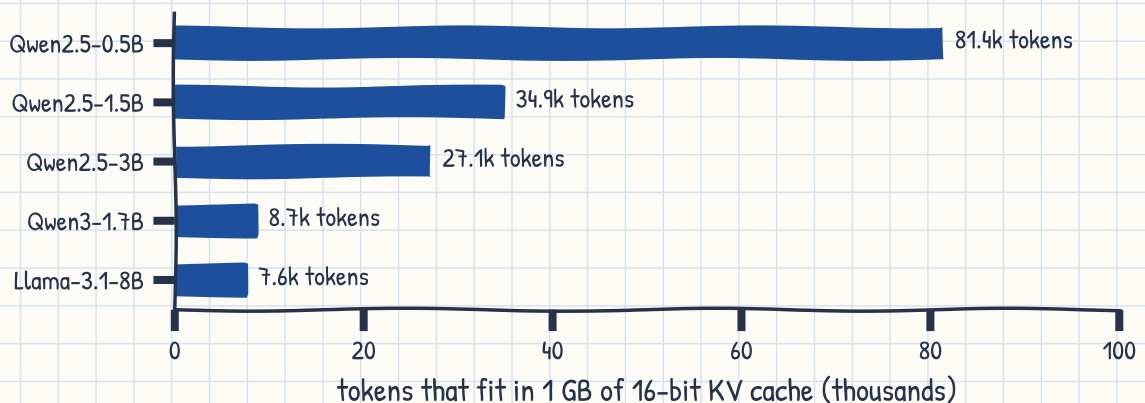
$$KV = 2 \times L \times H_{kv} \times D \times \text{bytes}_{\text{per element}}$$

$L = \text{layers}$ · $H_{kv} = \text{KV heads}$ · $D = \text{head dim}$ · the leading 2 is K and V

$$\text{Qwen2.5-1.5B: } 2 \times 28 \times 2 \times 128 \times 2 \text{ B} = 28,672 \text{ B} = 28 \text{ KiB}$$

My arithmetic from the published configs. Recompute them yourself from `config.json` — you will be asked to.

Model	Layers	KV heads × head dim	KV / token (16-bit)	Weights (BF16)
Qwen2.5-0.5B	24	2 × 64	12 KiB	≈ 1.0 GB
Qwen2.5-1.5B	28	2 × 128	28 KiB	≈ 3.1 GB
Qwen3-1.7B	28	8 × 128	112 KiB	≈ 3.4 GB
Qwen2.5-3B	36	2 × 128	36 KiB	≈ 6.2 GB (does not fit)
Llama-3.1-8B	32	8 × 128	128 KiB	≈ 16 GB (does not fit)



same 1 GB of VRAM, very different concurrency — pure arithmetic from the table above

► Fragmentation

- Reserving a contiguous `max_len` region per sequence wastes $(\text{max_len} - \text{actual})$ tokens for every sequence.
- Paging cuts the waste to less than one block per sequence.

7.4 VRAM budget (Qwen2.5-1.5B, BF16)

≈ 6 GiB on the card — estimates, replace with `torch.cuda.mem_get_info()` + `max_memory_allocated`



↑ logits, last position only ≈ 39 MB

classic mistake: logits for every prompt position = $2,048 \times 151,936 \times 4 \text{ B} \approx 1.24 \text{ GB}$ — most of the KV pool

where the 6 GB goes — estimates until you measure them

Item	Estimate	Measure with
Free after CUDA context	~5.5-5.8 GB of ~6 GiB	<code>torch.cuda.mem_get_info()</code>
Weights	≈ 3.1 GB	Parameter bytes
Activation peak at a 2,048-token step budget	0.1-0.3 GB	Profiling forward + <code>max_memory_allocated</code>
Logits, last position only (64 seqs × 151,936 vocab × FP32)	≈ 39 MB	—
Safety margin	0.2-0.3 GB	—
KV pool	≈ 1.5-2 GB ≈ 50-70k tokens	—

the classic mistake

Computing logits for every prompt position in your own forward pass. For a 2,048-token prompt that is $2,048 \times 151,936 \times 4 \text{ bytes} \approx 1.24 \text{ GB}$ — most of your KV pool.

7.5 Latency metrics and how they relate

Metric	Definition
<u>TTFT</u>	queue wait + tokenization + prefill steps + first sample + transport
<u>ITL</u>	gap between consecutive tokens, pooled across all tokens. The metric that shows stalls.
<u>TPOT</u>	per request: $(\text{E2E} - \text{TTFT}) \div (\text{n_out} - 1)$. It is an average, so it hides stalls.
<u>E2E</u>	$\approx \text{TTFT} + (\text{n_out} - 1) \times \text{TPOT}$
<u>Throughput</u>	must always say which kind: input tok/s, output tok/s or req/s
<u>Goodput</u>	requests per second that meet both SLOs

Little's law – sanity-check the harness with it

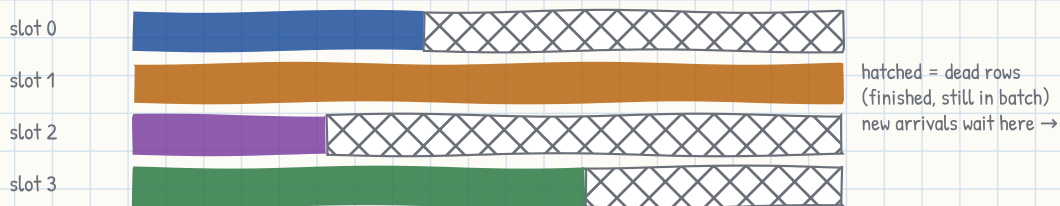
$$L = \lambda W$$

mean requests in the system = arrival rate × mean latency

7.6 The batching family

Kind	What decides batch membership
Static batching	A fixed batch runs to completion. Per request.
Dynamic batching	Collect up to B requests or wait W ms, then run. Still per request.
Continuous batching	Membership changes every step (Orca).
Chunked prefill	Prompts split into chunks and co-scheduled with decodes under a per-step token budget (Sarathi-Serve).

static batch: a batch lives as long as its longest request



continuous batch: membership changes every step (Orca)

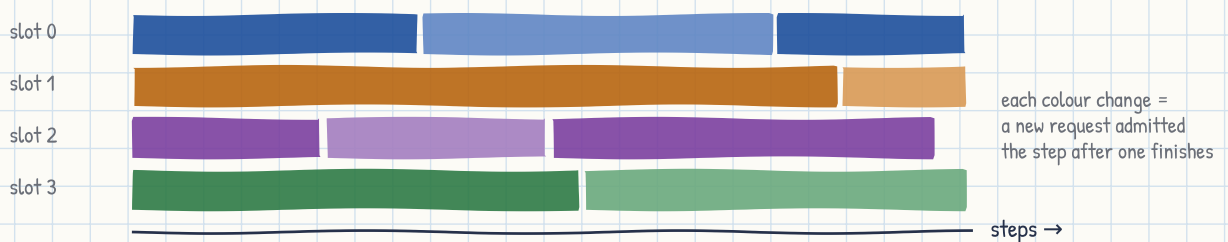


illustration – why iteration-level scheduling removes head-of-line blocking

7.7 Queueing

- Queue wait grows without bound as utilization approaches 1. For an M/M/1 queue it scales as $1 / (1 - \rho)$.
- The latency-throughput curve therefore has a **knee**. Goodput is measured to the left of it.
- With an unbounded queue under overload, everyone times out and goodput falls to zero. That is why admission control exists.

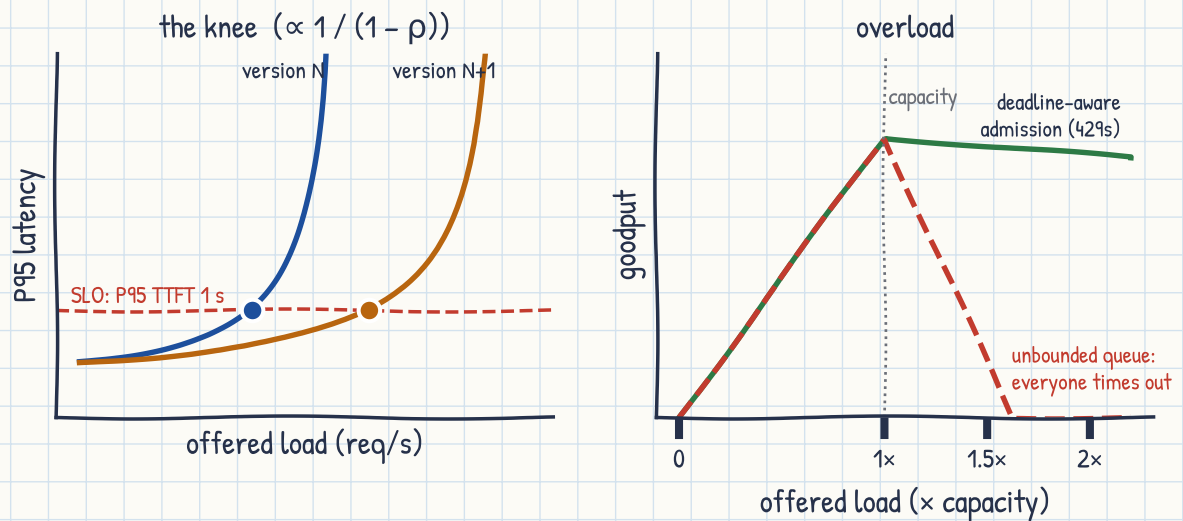


illustration of the shapes E5 and E10 measure — markers = goodput capacity at the SLO

7.8 The CUDA execution model

- Kernel launches are asynchronous. The CPU runs ahead until something forces a sync.
- If CPU time per step exceeds GPU time per step, the GPU sits idle between kernels.
- Caching allocator: reserved memory can far exceed allocated memory; that gap is fragmentation.
- Pinned memory is required for truly asynchronous copies.
- CUDA graphs record a kernel sequence with fixed shapes and addresses, then replay it with a single launch.

hidden sync points — hunt these down

`.item()` · `.tolist()` · `.cpu()` · printing a tensor · `if tensor:` · boolean-mask indexing ·
`torch.nonzero` · H2D copies from pageable memory

7.9 Numerics

- Use BF16; Qwen was trained in it. Keep RMSNorm, softmax and logits in FP32.
- Changing the batch composition changes reduction order. That causes tiny logit differences, and greedy outputs can diverge after many tokens.
- That divergence is expected. Measure how often and how early it happens (E17), so you can tell it apart from a real bug.

Reading list

Read these before building:

- ☐ Orca (continuous batching)
- ☐ Sarathi-Serve (chunked prefill)
- ☐ vLLM / PagedAttention
- ☐ SGLang (RadixAttention)

- ☐ DistServe (goodput; prefill/decode disaggregation)
- ☐ FlashAttention 1 and 2
- ☐ Flash-Decoding (split-KV decode)
- ☐ Horace He, "Making Deep Learning Go Brrrr From First Principles"
- ☐ PyTorch's gpt-fast blog post

Read vLLM's scheduler and block-manager source only after you have written your own first version.

8 Baseline system (V0)

Deliberately simple, fully instrumented. V0 is the yardstick; its job is to be measured, not to be fast.

What to build

- FastAPI endpoint `/v1/completions`, an OpenAI-compatible subset:
 - fields: `prompt`, `max_tokens`, `temperature`, `top_p`, `seed`, `stream`, plus an `ignore_eos` extension;
 - SSE format `byte-compatible with vLLM`, so one harness drives both.
- Hugging Face `AutoModelForCausalLM`, Qwen2.5-1.5B-Instruct in BF16 with SDPA attention.
- One worker thread and a global lock, so exactly one request is served at a time.
- `model.generate()` with `TextIteratorStreamer`.
- An unbounded wait queue. **This is deliberate; E10 shows why it is bad.**
- No warm-up. Then add warm-up as a one-line V0.1 change and measure the difference in first-request latency.
- Full instrumentation from day one:
 - request ids and timestamps: received, tokenized, started, first token, finished;
 - Prometheus histograms, JSON logs, the NVML sampler.

why instrument the baseline so heavily?

The harness only talks HTTP, so measurement never changes when the engine does. Every later version is compared through exactly the same lens.

Reference baselines – run them now, on the same traces

- vLLM: same model and `max_model_len`, with prefix caching on and off. You will probably need to lower `gpu-memory-utilization`, `max-num-seqs` and `max-model-len` to start it on 6 GB. Document the flags.
- llama.cpp server: GGUF of the same model at F16 and Q8_0.

These curves are the targets on your headline plot.

What VO measures

Measurement	Setup
Unloaded latency decomposition	$P \in \{128, 512, 1024, 2048\}$, $n_{\text{out}} = 256 \rightarrow$ tokenization, prefill and TPOT
Decode efficiency	measured TPOT vs. the bandwidth ceiling (16 ms/token)
Concurrency	closed-loop with 1, 2, 4, 8 and 16 users, plus an open-loop rate sweep
Cold start	process start $\rightarrow$ ready, and first-request vs. tenth-request latency

Hypotheses to check (not results)

- ☐ Throughput stays flat as concurrency rises.
- ☐ TTFT grows with queue position.
- ☐ `nvidia-smi` shows high utilization even though the work is about 1 FLOP/byte.
- ☐ TPOT sits some margin above the ceiling. That gap is framework and host overhead, which you will later locate precisely.

9 Optimization roadmap

Eight versions of one codebase. Each attacks a measured bottleneck and pays a stated cost.

Ver.	Change	Bottleneck attacked	Cost	Priority
V0	HF generate, serialized	— (measure)	—	must
V1	Own runtime; static and windowed batching	Weight reads per token at batch 1	TTFT from the window, padding, HOL blocking	must
V2	Continuous batching, slot-based KV	HOL blocking from output-length variance	ITL spikes from prefill; KV over-reservation	must
V3	Paged KV, admission, preemption, backpressure	Fragmentation caps concurrency; overload collapse	Indirection, rejections, recompute waste	must
V4	Prefix caching	Redundant prefill of shared prefixes	Cache memory vs. concurrency; hashing CPU	must
V5	Chunked prefill, token budget, cost-model admission	Prefill-decode interference	TTFT of long prompts	must
V6	CUDA graphs, persistent buffers	Host overhead, GPU idle gaps	Startup time, memory, bucket padding	should
V7	INT8/INT4 weights, FP8 KV	Bytes per step; capacity	Quality; speed depends on batch size	should

order of V6 and V7

Let the V5 profile decide. Whichever bottleneck is larger goes first. That decision, backed by a profile, is itself a piece of evidence of judgment.

V1 — Own the runtime, then batch per request

► Bottleneck

- At batch 1, every token streams about 3.1 GB of weights while doing about 1 FLOP per byte.
- `generate()` is a black box: you cannot batch mixed requests or control the loop.

► Hypothesis

1. Your forward pass matches HF logits within BF16 tolerance, and batch-1 TPOT is equal to or better than V0.
2. Decode step time stays nearly flat from $B = 1$ to $B \approx 16-32$, so output throughput scales nearly linearly with B .

3. Per-request batching then adds HOL blocking proportional to output-length variance, and a batching window adds up to W ms to TTFT.

► Implementation

Part A — the runtime

- `safetensors` loader that maps HF parameter names to your modules;
- Qwen2 layers;
- RoPE with precomputed cos/sin tables indexed by position ids;
- a preallocated contiguous KV cache `[L, 2, B_max, H_kv, S_max, D]`;
- explicit `prefill()` and `decode_step()` functions;
- `lm_head` on the last position only;
- a batched sampler.

Nothing is benchmarked until the correctness gate passes.

Part B — batching

- static: wait for B requests;
- windowed: B requests or W ms, whichever comes first;
- prompts are left-padded;
- per-row position ids must start at 0 at the first real token — getting this wrong is a classic bug;
- finished rows keep occupying the batch until all rows finish.

► Benchmark

- E2: decode step time vs. B — the "flat line" plot.
- Goodput vs. arrival rate for $W \in \{0, 10, 50, 200\}$ ms.
- Wasted-token fraction (padding plus post-finish rows) for fixed vs. lognormal output lengths.

► Cost

TTFT rises with the window; compute is wasted on padding and finished rows; HOL blocking.

► What you learn

- Why batching raises utilization, and why `nvidia-smi` cannot show it.
- Why batching hurts TTFT.
- How padding interacts with positions and masks.
- How to validate a model implementation numerically.

V2 – Continuous (iteration-level) batching

► Bottleneck

A per-request batch lives as long as its longest request. With lognormal output lengths, most rows are dead most of the time, and new arrivals wait for the whole batch.

► Hypothesis

- Iteration-level scheduling removes HOL blocking.
- P95 E2E and goodput improve most when output-length variance is high, and barely at fixed lengths.
That fixed-length run is your control.
- Two new problems will appear:
 - ITL spikes whenever a prefill runs, because decodes wait for it;
 - concurrency is still capped at $B_{\max} \times S_{\max}$ reserved tokens, even though most slots are mostly empty.

► Implementation

- An engine loop with WAITING and RUNNING states.
- Each step first admits new requests into free slots and runs their prefill as its own forward pass. This is prefill-first; make decode-first a policy flag too.
- Then one decode step for all running sequences.
- Per-slot sequence lengths. Finished sequences leave immediately.
- The abort path, and streaming driven by engine outputs.
- Note: a reference SDPA over $S_{\max}$ with a length mask still reads the full reserved length. Measure that cost.

► Benchmark

- E4: comparison of batching policies.
- An ITL timeline aligned with the step trace, showing spikes on prefill steps.
- KV reservation efficiency over time: $\sum \text{actual tokens} \div (B_{\max} \times S_{\max})$.

► Cost

More host work per step. Prefill-first trades ITL for TTFT; decode-first does the reverse.

► What you learn

Orca's core idea, and where prefill-decode interference comes from.

V3 – Paged KV, memory-aware admission, preemption, backpressure

► Bottleneck

- Reserving $S_{\max}$ tokens per sequence means KV memory, not compute, caps concurrency.
- The queue is unbounded, so overload becomes timeouts for everyone.

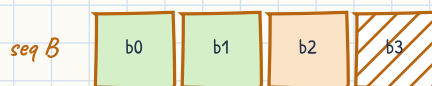
► Hypothesis

- With 16-token blocks, waste falls below one block per sequence.
- Maximum concurrent sequences in the same VRAM therefore rises by roughly $S_{\max} \div \text{mean actual length}$.
- Goodput at high load rises until bandwidth, rather than memory, limits step time.
- Reference gather-based attention costs extra bandwidth that grows with context, which motivates a real paged kernel.
- With admission control, goodput under 2× overload stays near capacity instead of collapsing.

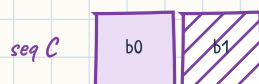
logical blocks (16 tokens each)



► [7, 2, 11]



► [7, 2, 5, 14]



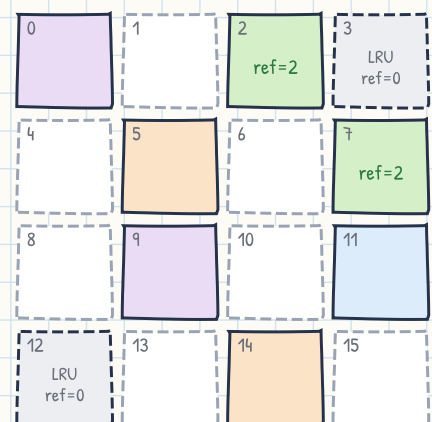
► [9, 0]

A and B share a system prompt:
their first two blocks (green) map
to physical 7 and 2 (refcount 2).

the hatched last block is partial:
never shared, returned to the free
list on finish — so no copy-on-write.

invariant (debug mode):
 $\text{free} + \text{in_use} + \text{evictable} == \text{total}$

physical KV pool [num_blocks, H_kv, 16, D]



white dashed = free · grey = evictable (cached, ref 0)

a sequence needs < 1 block of waste

block tables map logical blocks to scattered physical blocks — shared prefixes are just refcount > 1

► Implementation

KV pool

Per-layer pool [num_blocks, H_kv, block_size, D] for K and V. num_blocks comes from free memory after a profiling forward pass at the maximum token budget, minus a margin.

Allocator and writes

- BlockAllocator with free list and refcounts;
- per-sequence block tables;
- slot_mapping scatter writes (index_copy_, or a small Triton kernel).

Reference paged decode attention

Gather blocks into `[B, H_kv, max_ctx, D]`, apply a length mask, run SDPA. Test it against contiguous attention using shuffled, non-contiguous block tables.

Admission

Admit only if free plus evictable blocks $\geq$ blocks needed for the prompt plus a watermark (1-5% of the pool, reserved so running decodes can grow).

Preemption (recompute)

When a running sequence cannot get a block, preempt the most recently admitted sequence and requeue it at the head.

Backpressure

- bound the waiting queue by queued tokens;
- the engine publishes a throughput estimate;
- the frontend returns 429 with `Retry-After` when the predicted wait exceeds the deadline.

stretch: a Triton split-KV paged decode kernel

- Parallelism in naive decode attention is batch $\times$ KV heads. At batch 1 with 2 KV heads that is 2 thread blocks on a 20-SM GPU, so you must also split along the sequence (flash-decoding) and reduce with log-sum-exp.
- Process all 6 query heads of a GQA group in one program, so each K/V block is loaded once.

► Benchmark

- E6: maximum concurrency, and a block-size sweep.
- E7: attention kernels.
- E10: overload with and without admission control.
- Preemption rate vs. load.

► Cost

- Indirection: block-table lookups and non-contiguous reads.
- The block-size trade-off: large blocks mean internal fragmentation and coarse prefix sharing; small blocks mean bigger tables and less efficient memory access.
- Rejections — the explicit price of bounded latency.
- Recompute waste.

► What you learn

Paged attention; memory as the binding constraint of serving; how admission control and backpressure actually behave.

V4 – Prefix caching

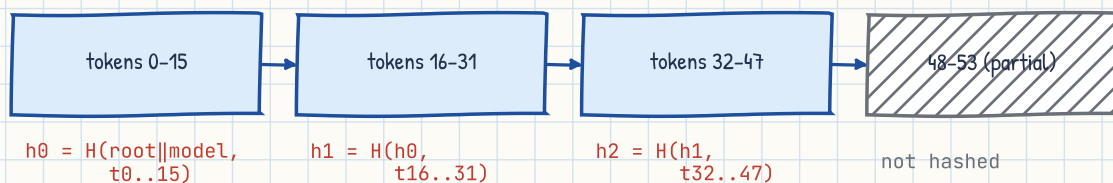
► Bottleneck

Real traffic repeats prefixes: system prompts, few-shot templates, multi-turn history. V3 recomputes them every time, in the most expensive, compute-bound phase.

► Hypothesis

- With a cached fraction f of the prompt, TTFT drops by nearly f of the prefill compute. Slightly less, because suffix queries still attend over the cached prefix.
- Hit rate falls off a cliff once the working set of distinct prefixes exceeds evictable capacity.
- Under memory pressure, cached blocks compete with live sequences, so **caching can reduce concurrency**.

each block's key includes its parent's hash $\rightarrow$ equal key $\Rightarrow$ equal entire prefix (up to collisions; 128-bit)



lookup = walk the chain until the first miss · full-prompt hit $\rightarrow$ recompute the last token for logits

chained block hashes: a key identifies the whole prefix, not just one block

► Implementation

Hashing

- hash full blocks only: $h_i = H(h_{i-1}, \text{block}_i \text{ tokens})$;
- use a 128-bit hash, and be ready to discuss collision probability;
- include the model identity in the root hash.

Lifecycle

- look up at admission and share hits via refcount;
- finished sequences insert their full blocks;
- blocks at refcount 0 move into the LRU evictable set;
- the allocator evicts from the LRU when the free list is empty.

The "prefill over cached KV" path

Suffix queries attend to gathered prefix KV plus suffix KV, with a **lower-right causal mask**.

Edge cases

- a full-prompt hit recomputes the last token;
- the last partial block is never shared, so you need no copy-on-write. Be able to explain why.

► Benchmark

- E8: TTFT vs. shared-prefix fraction $\times$ prefix length.
- Hit rate vs. number of distinct prefixes, to find the thrash cliff.
- A multi-turn trace.
- Hashing CPU cost per request.
- Correctness: greedy outputs must match between hit and miss.

► Cost

- Cache memory competes with concurrency.
- Hashing and bookkeeping CPU cost.
- If you reorder the queue to favour cache hits, requests that miss the cache see worse tail latency. Measure it.

► What you learn

Why prefix caching is block-granular; hash chain vs. radix tree; cache locality. At multi-replica scale, cache locality becomes the routing problem.

V5 – Chunked prefill, token budget, cost-model admission

► Bottleneck

- A 2k-token prefill takes a whole step of roughly 0.3–0.4 s (estimate). Every running stream stalls during it.
- P99 ITL is therefore set by **other users'** prompts.
- Meanwhile ordinary decode steps are memory-bound and leave compute idle.

illustration — box width = step duration (shape only, not measured)

V4 — whole prefill as one step

 decode-only step
 decode + prefill chunk



2k-token prefill $\approx 0.3\text{--}0.4$ s (estimate): every stream stalls $\rightarrow$ P99 ITL spike

V5 — chunked prefill, token budget T per step



decodes first, then prefill chunks fill the rest of the budget $\rightarrow$ bounded step time

illustration — one giant prefill step vs. prefill spread across bounded steps

► Hypothesis

- Cap each step at T tokens: decodes first, remaining budget filled with prefill chunks. That bounds step time and therefore ITL.
- Decode steps are already streaming the weights, so a piggybacked prefill chunk costs little extra time.
- Throughput should hold or improve while P99 ITL drops sharply.
- The TTFT of long prompts rises, because their prefill is spread across steps.

► Implementation

- The V4 "prefill over cached KV" primitive already handles a chunk attending to earlier chunks.
 Chunked prefill is mostly a scheduler change.
- Mixed batch: dense layers run over [decode tokens || chunk tokens]; attention dispatches per group.
- The fitted cost model picks T dynamically, so predicted step time stays within the TPOT SLO for the current decode load.

► Benchmark

- E9: $T \in \{64, 128, 256, 512, 1024, \infty\}$ under bursty long-prompt traffic. Plot the TTFT-P95 vs. ITL-P99 Pareto frontier.
- Goodput vs. rate, V4 vs. V5.
- Cost-model prediction error.

► Cost

TTFT vs. ITL — the fundamental scheduling dial. Earlier chunks' KV is re-read by later chunks.

► What you learn

- Sarathi-Serve's insight.
- How to turn an SLO into a scheduler parameter.

- Why "prefill and decode have different bottlenecks" leads to co-scheduling at small scale and disaggregation at large scale.

V6 – Remove host overhead

► Bottleneck (confirm it with a profile — do not assume it)

- A decode step in eager PyTorch launches roughly 20 kernels per layer $\times$ 28 layers. That is several milliseconds of launch overhead before any scheduling work.
- Whether that overhead is exposed depends on the hardware ratio:
 - a 1.5B decode step's $\sim 16+$ ms of GPU time may hide it;
 - a 0.5B model's ~ 5 ms probably will not.
- Measure both models.

► Hypothesis

- CUDA graphs cut host time to close to one graph launch per step.
- The gain is largest for small models and small batches, and near zero when steps are bandwidth-saturated.
- The cost model sees the gain as a drop in α .

► Implementation

- Persistent device and pinned-host buffers for every step input; one H2D copy.
- Decode-only graphs captured for batch buckets {1, 2, 4, 8, 16, 32, 48, 64}, with block-table width fixed at `max_blocks_per_seq`. Batches are padded up to the next bucket.
- Mixed steps run eager. Optionally compare against `torch.compile`.
- Graph capture happens during warm-up; readiness waits for it.
- Stretch: overlap scheduling of step $t+1$ with GPU execution of step t .

► Benchmark

- E12: step time eager vs. graphs, across model size $\times$ batch.
- GPU idle fraction from Nsight Systems.
- Startup time and graph memory.
- Bucket padding waste.

► Cost

Startup latency, graph-pool memory, rigid shapes and addresses, harder debugging, padded compute.

► What you learn

Asynchronous execution; why the same code is GPU-bound on one card and CPU-bound on another.

V7 – Quantization: weights and KV

► Bottleneck

Bytes. Decode time $\approx$ bytes moved $\div$ bandwidth, and VRAM caps both model size and KV pool.

► Hypothesis

Weight-only INT8

- It approaches 2 $\times$ on the weight portion of batch-1 decode, but only if dequantization happens in registers inside the matmul.
- Naive "dequantize to BF16, then matmul" will be slower than plain BF16, because it writes and re-reads a full-precision copy.
- As batch grows, the dequantization arithmetic stops being free and the advantage shrinks. Find the crossover.

INT4 and FP8 KV

- INT4 lets Qwen2.5-3B fit with a usable KV pool.
- FP8 (E4M3) KV roughly doubles the tokens that fit in the pool, at a measurable quality cost.

► Implementation

- INT8 per-output-channel quantizer.
- Naive dequantization path.
- Fused path: a Triton dequant-GEMV kernel, or torchao weight-only INT8 with `torch.compile`.
- INT4 via an AWQ/GPTQ checkpoint and a library kernel. That is fine here: you are testing a hypothesis about bytes, not proving GEMM skill.
- FP8 KV with per-head scales, dequantized inside attention.

► Benchmark (E13)

- Speed: decode tok/s at batch 1 and 32; VRAM breakdown; maximum concurrency.
- Quality is mandatory:
 - perplexity on a fixed held-out text set;
 - greedy agreement with BF16 on 200 prompts;
 - exact-match accuracy on about 250 GSM8K problems.

► Cost

Quality vs. VRAM vs. speed; speed crossovers that depend on batch size.

► What you learn

For decode, quantization is a bandwidth and capacity optimization, and it has to be evaluated honestly.

10 Repository structure

```

ridgepoint/
├── ridgepoint/
│   ├── README.md
│   ├── Makefile
│   ├── pyproject.toml, uv.lock # setup · test · test-gpu · bench-smoke · bench EXP=E05 · perf-gate · report
│   ├── docker/ # pinned environment
│   ├── docker/ # CUDA base image with pinned wheels; compose for Prometheus + Grafana
│   ├── configs/
│   │   ├── models/ # qwen2.5-0.5b, qwen2.5-1.5b, qwen3-1.7b, qwen2.5-3b-int4
│   │   ├── engine/ # v0.yaml ... v7.yaml – each version is a set of flags on one codebase
│   │   ├── workloads/ # chat, prefill_heavy, decode_heavy, multiturn, burst, overload
│   │   └── slo.yaml
│   ├── src/ridgepoint/
│   │   ├── server/ # FastAPI app, OpenAI schemas, SSE, admission gate, detokenizer, health
│   │   ├── engine/ # engine loop, sequence states, output processor, supervisor, EngineClient
│   │   ├── ipc/ # message types (msgspec), in-process and ZMQ transports
│   │   ├── scheduler/ # static/windowed/continuous/chunked policies, budget, admission, preemption
│   │   ├── kvcache/ # block allocator, block tables, prefix cache, swap space
│   │   ├── runtime/ # model runner, input builder, CUDA graphs, sampler, weight loader
│   │   │   └── models/ # qwen2.py, qwen3.py, layers.py (RMSNorm, RoPE, MLP)
│   │   ├── kernels/ # attention: reference, triton_paged_decode, flashinfer adapter; KV write
│   │   ├── quant/ # INT8 per-channel, INT4 adapters, FP8 KV
│   │   ├── perfmodel/ # measured hardware constants, step-time model, fitting, validation
│   │   ├── telemetry/ # Prometheus metrics, step trace, NVML sampler, NVTX helpers, JSON logs
│   │   └── baseline/ # V0: HF generate() server, frozen
│   ├── bench/
│   │   ├── workloads/ # length dists, arrival processes, prefix groups, sessions → traces
│   │   ├── client/ # open- and closed-loop streaming load generator
│   │   ├── backends/ # launch configs: Ridgepoint vN, vLLM, llama.cpp, HF, null backend
│   │   ├── micro/ # bandwidth, GEMM, launch overhead, PCIe, attention kernels, tokenizer
│   │   ├── sweep.py # experiment matrix → results/<run_id>/ with manifest
│   │   └── analyze/ # per-request tables, percentiles + bootstrap CIs, goodput, plots
│   ├── experiments/ # E01_hardware ... E17_numerics: README, config, plots
│   ├── profiling/ # nsys / ncu / torch-profiler / memory-snapshot scripts + FINDINGS.md
│   ├── tests/ # unit/ numerics/ integration/ concurrency/ stress/ perf/ (baselines/*.json)
│   ├── sim/ # stretch: discrete-event router simulator, calibrated by perfmodel
│   ├── dashboards/grafana/ # one dashboard JSON
│   └── docs/
│       ├── report.md # the engineering report
│       ├── architecture.md
│       ├── adr/ # 0001-single-writer-engine, 0002-no-redis, 0003-block-size, ...
│       ├── memory-budget.md
│       ├── benchmarking-methodology.md
│       └── failure-modes.md

```

results hygiene

Raw results stay out of git. Summarized JSON and plots are committed.

Every result links to the git SHA and config hash that produced it.

► Where to look first in this tree (reviewer's eye)

- `src/ridgepoint/kvcache/` — refcounting, eviction, invariants.
- `src/ridgepoint/scheduler/` — the step decision, testable without a GPU.
- `bench/` — methodology: open-loop load, warm-up exclusion, raw-sample percentiles.
- `docs/report.md` — the engineering story with costs and negative results.

11 Benchmarking framework

The harness is the product. If it is wrong, every number in the repo is wrong.

Workload specification

configs/workloads/chat_shared_prefix.yaml

```
name: chat_shared_prefix
seed: 1234
duration_s: 300
warmup_s: 30
arrival: {process: poisson, rate_rps: 2.0} # excluded from metrics
prompt_len: {dist: lognormal, median: 300, sigma: 0.8, min: 16, max: 2048} # or {process: closed, users: 16} or a trace file
output_len: {dist: lognormal, median: 200, sigma: 0.7, min: 8, max: 1024}
ignore_eos: true # output length is a controlled variable
shared_prefix: {num_prefixes: 4, prefix_len: 400, fraction: 0.5, popularity: zipf, zipf_s: 1.1}
sessions: null # or {turns: [3, 8], think_time_s: {dist: exp, mean: 5}}
sampling: {temperature: 0.0}
slo: {ttft_p95_ms: 1000, tpot_p95_ms: 75}
```

► Standard workloads

Name	Shape	What it stresses
<u>W-chat</u>	the spec above	everything, realistically
<u>W-prefill</u>	1.5-3k-token prompts, 64-256-token outputs	compute, TTFT, prefill interference
<u>W-decode</u>	64-256-token prompts, 512-1024-token outputs	bandwidth, KV growth, preemption
<u>W-multiturn</u>	sessions with think time	prefix cache reuse
<u>W-burst</u>	W-chat plus on/off bursts of long prompts	chunked prefill, ITL tails
<u>W-overload</u>	W-chat at 1.5× and 2× measured capacity	admission control, recovery

Harness components

1. Trace generator.

- Writes JSONL: req_id, t_offset_ms, prompt_ids, max_tokens, prefix_group, session.
- Prompts are cut to exact token lengths from real text.
- Seeded, so every version and every backend replays the identical trace.
- Send token ids wherever the backend accepts them. Always record the server-reported prompt token count, and reject the run if it deviates.

2. Replayer (load generator).

- asyncio with httpx streaming.

- **Open-loop by default**: each request is sent at its scheduled time regardless of completions. Closed-loop benchmarks slow down when the server does, which hides overload (coordinated omission).
 - Records `perf_counter_ns` for every SSE event, and counts tokens from server-supplied counts, never by re-tokenizing text.
 - Pinned to different CPU cores from the server (`taskset`).
 - Measures its own lag: actual send time minus scheduled send time.
3. Null backend. A fake server that streams tokens at a configured rate. It proves the generator can drive 500+ concurrent streams before you trust it to measure anything.
 4. Server-side timing. The final SSE event carries queue, prefill and decode times, cached tokens and preemption count. The step trace joins to requests by `request_id`.
 5. Collectors. A Prometheus snapshot at run start and end, NVML at 10 Hz, and the step trace.
 6. Analysis. A per-request table (parquet). Percentiles computed from raw samples — **never average percentiles across runs**. Bootstrap confidence intervals, goodput, plots.
 7. Run manifest. Git SHA, config hash, model hash, torch/CUDA/driver versions, GPU name, power limit, clock and temperature traces, and workload seed.

*validity rules — a run is **INVALID** if any hold*

- ✗ generator lag P99 > 5 ms;
- ✗ SM clock more than 10% below the run median for more than 5% of the run;
- ✗ any error the experiment did not intend;
- ✗ too few requests in the measurement window;
- ✗ Little's-law mismatch > 10%.

Statistics

- P99 needs sample size. Measure it only with at least 1,000–2,000 requests per point. **From 200 requests, P99 is two samples.**
- Trials: at least 3 per point with different seeds. Report the median and the spread.
- Cache state: state explicitly whether each prefix-cache experiment starts with a cold or warm cache.

The headline plot

what goes on it

- One plot per workload.
- X axis: offered load (req/s).
- Two panels: P95 TTFT and P95 TPOT, with SLO lines drawn.
- One curve per version, plus vLLM and llama.cpp.
- A marker at each curve's goodput capacity.

Metrics that matter

Metric	Definition as computed	Why it matters
TTFT P50/P95/P99	First-token arrival – send (client side)	Interactivity; dominated by queue + prefill
ITL P50/P99	Consecutive-token gaps, pooled across all tokens	Stream smoothness; exposes prefill and preemption stalls
TPOT P95	$(E2E - TTFT) / (n_{out} - 1)$ per request	Per-user generation speed; the SLO metric
E2E P95	Last token – send	Non-streaming and batch users
Queue time	Scheduled – enqueued (server side)	Separates scheduling delay from compute
Prefill / decode time	Per request, server side	Attribution
Output tok/s, input tok/s	Over the measurement window	Efficiency; always say which
Goodput	Req/s meeting both SLOs	The headline number
Rejection rate	$4xx/5xx \text{ by reason} \div \text{offered}$	The price of admission control; always shown next to goodput
KV utilization	Used / evictable / free blocks	Memory pressure
Prefix hit ratio	$\text{Cached prompt tokens} \div \text{prompt tokens}$	<u>Must be token-weighted</u> ; request-weighted ratios mislead
Preemptions/s, wasted tokens	Recomputed + aborted tokens	Hidden inefficiency
Step time P50/P99, GPU idle fraction	Step trace, Nsight Systems	Host overhead, jitter
MBU (model bandwidth utilization)	$\text{Modeled bytes per decode step} \div \text{GPU step time} \div \text{measured peak bandwidth}$	The real efficiency metric for decode
NVML util, VRAM, SM clock, power, temp	NVML	Context and run validity. Utilization only means "a kernel was running", not efficiency

12 Experiments

Each experiment gets a folder under `experiments/` with a README in a fixed shape:

Question → Hypothesis (with a predicted number) → Setup (hardware, model, workload, config, trials) → Result (table/plot with CIs) → Interpretation → Cost → Next bottleneck

Keep failed hypotheses in the report. ★ = one of the core ten: E1, E2, E3, E5, E6, E8, E9, E10, E15, E17.

E1 – Hardware characterization ★

- measure*
- DRAM bandwidth (large-tensor copy);
 - BF16 GEMM TFLOPS across shapes;
 - kernel launch overhead;
 - pinned vs. pageable H2D bandwidth;
 - host DRAM bandwidth;
 - how all of these change when the laptop throttles.

output Your ridge point and every ceiling in the project.

E2 – Two regimes ★

- setup* Prefill step time vs. tokens (64 → 4096); decode step time vs. batch (1 → 128) at fixed context.
- predict* Prefill time is linear in tokens beyond a small knee; decode time is flat until KV bytes become comparable to weight bytes.
- proves* Prefill is compute-bound and decode is memory-bound — measured rather than asserted.

E3 – Context length and full-pool step time ★

- setup* Decode step time vs. context (128 → 8k) at batch {1, 8, 32}, for Qwen2.5-1.5B vs. Qwen3-1.7B.
- predict*
- step time $\approx (W + \sum KV) / BW$;
 - for Qwen2.5-1.5B, KV bytes never exceed about half the weight bytes within its pool;
 - Qwen3-1.7B reaches its concurrency limit about 4× sooner.
- proves* Why GQA and KV size determine throughput.

E4 – Batching policy under output-length variance

setup Static vs. windowed vs. continuous; fixed vs. lognormal outputs.
measure Goodput and wasted-token fraction. The fixed-length run is the control.

E5 – Latency-throughput curves and goodput (the headline) ★

setup Every version, plus vLLM and llama.cpp, on W-chat, W-prefill and W-decode.
also A sensitivity analysis to SLO tightness. Close with a written gap attribution against vLLM.

E6 – Memory to concurrency ★

measure Maximum concurrent sequences before the first preemption, slot-based vs. paged.
sweep Block size {8, 16, 32, 64}: internal fragmentation vs. attention kernel time vs. prefix-sharing granularity.

E7 – Paged decode attention kernels

setup Gather-reference vs. Triton split-KV vs. FlashInfer vs. contiguous SDPA, across context lengths and batch sizes.
report Achieved bandwidth as a percentage of the measured peak.
predict The gather path costs about 2× the bytes of a direct read.

E8 – Prefix caching ★

setup Shared fraction {0, 25, 50, 75, 90}% × prefix length {256, 1024, 2048}.
also Hit rate vs. number of distinct prefixes (the thrash cliff); a multi-turn trace; hashing cost per request.

E9 – Chunked-prefill Pareto frontier ★

setup Token budget sweep under W-burst.
plot TTFT-P95 vs. ITL-P99, plus throughput at each point.

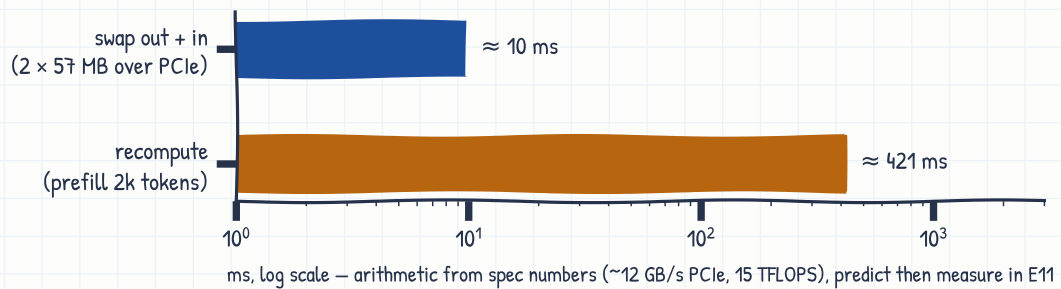
E10 – Overload, admission control and recovery ★

setup 2 minutes at 1.5× and 2× capacity, then back to normal load. Unbounded queue vs. deadline-aware admission.
measure Goodput, rejections, and time to drain the queue after the overload ends.
predict With an unbounded queue, goodput falls toward zero.

E11 – Preemption: recompute vs. swap

idea Recompute costs a prefill of the whole context; swap moves the KV over PCIe twice.

predict For Qwen2.5-1.5B at 2k context: swap moves about 57 MB, a few ms each way; recompute is about 6 TFLOP, several hundred ms. So **swap should win by well over an order of magnitude** — roughly 25-40× counting both directions, depending on achieved FLOPS and PCIe bandwidth. For an MHA 7B model on an H100 the same arithmetic comes out close to even.



task Predict, measure, and explain why the answer flips with hardware and model. Also measure how prefix-cache retention makes recompute cheaper.

E12 – Host overhead

- (a) Eager vs. CUDA graphs × model size (0.5B vs. 1.5B) × batch size.
- (b) In-process engine vs. separate engine process under 200 concurrent streams. Measure step-time jitter (P99 – P50) and ITL P99.

E13 – Quantization matrix

matrix Weights: BF16, INT8 naive, INT8 fused, INT4. KV: BF16, FP8.

measure Decode tok/s at batch 1 and 32, maximum concurrency, VRAM and quality. Find the batch size where weight-only quantization stops helping.

E14 – When the model does not fit

setup Llama-3.1-8B 4-bit in llama.cpp, sweeping `n_gpu_layers`.

predict tok/s with a two-tier bandwidth model: GPU bytes ÷ GPU bandwidth + CPU bytes ÷ host bandwidth.

why Validates your bandwidth reasoning in an engine you did not write.

E15 – Performance-model validation ★

method	Fit α , BW_eff and F_eff from the step trace on one set of operating points, and validate on held-out points.
plot	Predicted vs. measured, and the error distribution. Say where the model breaks and why.

E16 – Cold start and warm-up

break down	Time to ready: imports; CUDA context; weight load (disk → host → GPU); profiling run and KV allocation; graph capture.
compare	First-request latency with and without warm-up.

E17 – Numerical equivalence and batch invariance ★

measure	Per-layer differences vs. HF, and greedy agreement. Distribution of the first divergence position, batch 1 vs. batch 32, BF16 vs. an FP32 reference.
---------	--

13 Failure-mode testing

Each card reads: failure → inject → detect → system response → signal → recovery / verification.

1 KV blocks exhausted mid-decode never cut

inject	Cap <code>num_blocks</code> ; long outputs
detect	Allocator cannot serve a running sequence's next block
response	Preempt newest-admitted (recompute or swap); requeue at head
signal	<code>preemptions_total</code> ; log with victim id
verify	Victim's greedy output equals an unpreempted run

2 CUDA OOM (activation spike)

inject	Ballast tensor shrinks free VRAM; oversized token budget
detect	<code>torch.cuda.OutOfMemoryError</code> around forward
response	Retry the prefill chunk at half size, or fail it; decodes continue; <code>empty_cache()</code>
signal	<code>oom_total{phase}</code>
verify	Engine keeps serving; soak test shows no drift

3 Prompt > `max_model_len` or > whole KV pool

inject	50k-token prompt
detect	Frontend, after tokenization
response	400 with reason; never enqueued
signal	<code>rejected_total{reason}</code>
verify	Others' latency unaffected

4 Large request cannot fit now (HOL blocking in admission)

inject	3k-token prompt under load
detect	Queue head unadmittable for k steps
response	Bounded skip-ahead plus aging
signal	<code>hol_skips_total</code> ; its queue time
verify	Starvation bound holds in test

5 Sustained overload *never cut*

inject	Open-loop 1.5x / 2x capacity
detect	Predicted wait > deadline
response	429 + Retry-After
signal	Rejections by reason; goodput
verify	Goodput near capacity; queue drains in bounded time

6 Client disconnect mid-stream *never cut*

inject	Clients drop after random k tokens
detect	Stream task cancelled / <code>is_disconnected()</code>
response	ABORT → removed at next step; blocks released
signal	<code>aborted_total</code> ; wasted tokens
verify	Blocks in use return to baseline (leak test)

7 Slow consumer

inject	Client sleeps between reads
detect	Per-request output queue full
response	Coalesce tokens into fewer events; abort after stall timeout
signal	<code>slow_consumer_aborts_total</code>
verify	Engine step time unaffected

8 Deadline exceeded

inject	Tight deadlines
detect	Checked every step, both queued and running
response	Abort with <code>finish_reason="timeout"</code>
signal	<code>timeouts_total{stage}</code>
verify	Blocks freed

9 Malformed request

inject	Hypothesis fuzzing: bad JSON, NaN temperature, <code>top_p</code> $\notin (0,1]$, <code>max_tokens</code> ≤ 0 or huge, invalid UTF-8, empty prompt, 10k stop strings
detect	Pydantic validation
response	422 with the offending field
signal	<code>invalid_requests_total</code>
verify	Nothing reaches the engine

10 Non-finite logits

inject	Force FP16 overflow in a test build
detect	Cheap <code>isfinite</code> check on sampled rows
response	Fail that sequence only
signal	<code>numeric_errors_total</code>
verify	Batch-mates unaffected

11 Engine crash / sticky CUDA error

inject	<code>kill -9</code> ; device-side assert in a debug kernel
detect	IPC EOF / missed heartbeats
response	In-flight requests fail with retryable 503; <code>/ready</code> = false; supervisor restarts engine
signal	<code>engine_restarts_total</code> ; time to ready
verify	Frontend survives; recovery time measured

12 Prefix-cache thrash

inject	Distinct prefixes > evictable capacity
detect	Hit ratio drops; evictions/s spikes
response	LRU eviction (compare policies)
signal	Hit ratio; <code>evictions_total</code>
verify	Degrades to no-cache performance, not below it

13 *Batch larger than largest CUDA graph bucket*

<i>inject</i>	Burst
<i>detect</i>	Size check
<i>response</i>	Eager fallback
<i>signal</i>	graph_fallback_total
<i>verify</i>	Output still correct

14 *Thermal / power throttling*

<i>inject</i>	Long run on battery
<i>detect</i>	NVML clocks and throttle reasons
<i>response</i>	Harness marks run invalid
<i>signal</i>	Clock trace
<i>verify</i>	Rerun

the distinction to know cold

- A CUDA OOM is recoverable inside the process.
- An illegal memory access leaves a sticky error in the CUDA context, and only a process restart clears it.
- That is the reliability argument for splitting frontend and engine processes.

Invariants checked after every failure test

- ☐ all requests reached a terminal state;
- ☐ `free + in_use + evictable = total` blocks;
- ☐ in-use blocks back to baseline;
- ☐ no running sequence without an owning stream;
- ☐ `memory_allocated` back to its post-warm-up baseline;
- ☐ step time back to normal.

14 Profiling strategy

Protocol – three canonical operating points

Point	Workload
OP-A	batch 1, 512-token context, decode
OP-B	32 sequences, ~1k context, decode only
OP-C	mixed prefill + decode under W-chat at 70% of capacity

Profile every version at all three. Record one paragraph of findings per profile in `profiling/FINDINGS.md`.

microbenchmarks first

E1's bandwidth, TFLOPS and launch-overhead numbers are the yardstick every profile is read against. A profile without a ceiling to compare to is just a picture.

nvidia-smi / NVML

shell

```
nvidia-smi --query-gpu=timestamp,utilization.gpu,memory.used,\
clocks.sm,clocks.mem,power.draw,temperature.gpu --format=csv -lms 100
```

Also log throttle (clock-event) reasons. The field name differs by driver version.

► Patterns to look for

- Utilization near 100% during batch-1 decode. NVML reports the fraction of time any kernel was running, not SM efficiency. Use MBU instead.
- `memory.used` includes the caching allocator's reserve. It is not live tensors.
- SM clock drops during long runs. That is thermal or power throttling.
- Clock locking (`nvidia-smi --lock-gpu-clocks`) reduces variance if your laptop driver permits it. If it does not, rely on the validity rules.

torch.cuda memory statistics

► What to track

- `memory_allocated` vs. `memory_reserved`. A large gap means fragmentation.
- `max_memory_allocated` from the startup profiling run. This sizes the KV pool.

- `num_alloc_retries` in `memory_stats()`. A retry means the allocator had to free cache and try again – a fragmentation warning.
- Snapshots: `torch.cuda.memory._record_memory_history(max_entries=100000)`, then `_dump_snapshot()`, viewed in PyTorch's memory visualizer.

► Patterns to look for

- Prefill memory peaks that scale with prompt length:
 - full-sequence logits (the 1.24 GB mistake);
 - FP32 upcasts in RMSNorm and softmax;
 - MLP intermediates at $8,960 \times$ tokens.
- The KV pool must be one allocation that never grows.
- Any per-step allocation in the decode loop is a bug once V6 lands.

PyTorch Profiler

Setup: use `schedule(wait, warmup, active)`, record shapes and stacks, and export a Chrome trace.

► Patterns to look for

- GPU time split between GEMMs, attention and elementwise ops.
- Kernels per step. Hundreds of tiny elementwise kernels are fusion candidates: RMSNorm, RoPE, SiLU-mul.
- Hidden syncs: `aten::_local_scalar_dense`, `cudaStreamSynchronize` or `cudaMemcpy` where you did not expect them.
- `aten::copy_` from pageable memory when building inputs.
- Growing `aten::cat` and `aten::index` time in the input builder as batch size rises.

Nsight Systems

► Setup

- Wrap every engine phase in an NVTX range: `schedule`, `build_inputs`, `forward`, `sample`, `output`.
- Capture a window of 20–50 steps with `--capture-range=cudaProfilerApi`, starting and stopping from Python.
- Trace `cuda,nvtx,osrt`.
- Enable GPU metrics sampling if your GPU and driver support it (the flag name varies by nsys version).
- Use `--cuda-graph-trace=node` when inspecting graphs.

► Patterns to look for

You see	It means
Gaps in the GPU row while NVTX shows <code>schedule</code> or <code>build_inputs</code>	The engine is host-bound
CUDA API row dominated by <code>cudaLaunchKernel</code>	Motivation for CUDA graphs. After V6: one <code>cudaGraphLaunch</code> per step
Where <code>cudaStreamSynchronize</code> sits	There should be exactly one per step
Pageable HtoD memcpy	Input buffers are not pinned
Long prefill kernels with decode streams waiting behind them	The V2 ITL spike, visible directly
Step jitter correlated with frontend-thread activity	GIL contention (E12b)
High DRAM bandwidth, low SM activity	Healthy memory-bound decode
Low bandwidth <u>and</u> low SM activity	Latency- or launch-bound

Nsight Compute

shell

```
ncu --set full -k regex:<kernel> --launch-skip 10 --launch-count 5 python bench/micro/attention.py
```

- It needs access to performance counters: run as root, or set the driver option that lifts the admin restriction.
- What to read for decode GEMMs and your attention kernel: memory throughput as a percentage of peak (Speed of Light), achieved occupancy, L2 hit rate and the roofline chart.
- What to expect: cuBLAS decode GEMMs should run at a high fraction of DRAM peak. If your attention kernel shows low bandwidth with low occupancy, you have the 2-thread-block parallelism problem that split-KV fixes.

15 Tests

Unit tests (CPU; run in GitHub Actions)

- BlockAllocator: property-based tests with Hypothesis, running random alloc/free/share/evict sequences and checking the invariants.
- PrefixCache: chain hashing, partial blocks never shared, full hit leaves the last token uncached, eviction order.
- Scheduler: a deterministic simulation with a fake executor whose step times come from the performance model. It checks:
 - the token budget is respected;
 - decodes are prioritized;
 - the preemption victim is chosen correctly;
 - the starvation bound holds;
 - aborts apply at step boundaries.
- Sampler: temperature $\rightarrow 0$ equals argmax; top-p keeps the smallest set with probability mass $\geq p$; seeded runs are reproducible.
- Incremental detokenizer vs. full decode on random token streams, including multi-byte characters split across tokens.
- Request validation.

tests/unit/test_block_allocator.py – the shape of a property test

```
@given(ops=st.lists(st.sampled_from(["alloc", "free", "share", "evict"]), max_size=500))
def test_allocator_invariants(ops):
    a = BlockAllocator(num_blocks=64)
    for op in ops:
        apply_random(a, op)
        # seeded helper picks targets
        assert a.num_free + a.num_in_use + a.num_evictable == 64
        assert a.sets_are_disjoint()
        # every block in exactly one set
        assert all(rc >= 0 for rc in a.refcounts)
```

Numerics tests (GPU)

► Against Hugging Face

- Per-layer hidden states vs. HF via hooks, at BF16 tolerance, plus an FP32 run for a tighter bound.
- Greedy agreement on 200 prompts, and the distribution of first-divergence position.

► Paged attention vs. the contiguous reference

- shuffled, non-contiguous block tables;
- context length 1;
- block boundaries ± 1 ;
- partial last blocks;
- GQA grouping.

► Equivalence tests

- ☐ cache hit vs. miss;
- ☐ batch 1 vs. batched, within tolerance;
- ☐ chunked vs. whole prefill;
- ☐ graph vs. eager.
- ☐ preempted vs. uninterrupted;

the mask-alignment test

A query chunk shorter than its KV must fail with SDPA's `is_causal=True` (top-left aligned) and pass with a lower-right causal mask. If this test does not exist, chunked prefill and prefix caching are unverified.

Integration tests

- End-to-end streaming over HTTP.
- The OpenAI client and the vLLM benchmark client work unmodified.
- Finish reasons, the abort path, and `/live` and `/ready` transitions during warm-up and restart.

Concurrency tests

- Load: 500 concurrent streams for 10 minutes, with random disconnects, random deadlines and random sampling parameters.
- Invariants at the end: every request terminated, blocks back to baseline, no orphaned sequences.
- Event-loop check: run with asyncio debug mode, so blocking calls in the event loop surface as slow-callback warnings.

Stress and soak tests

- Load: one hour at 80% of measured capacity, with periodic overload bursts.
- Track: host RSS, `memory_reserved`, `num_alloc_retries`, and P99 drift over time.

Performance regression tests

- Traces: three canonical 3-minute traces: decode-heavy, prefill-heavy, shared-prefix.
- Baseline: built from 5 runs.
- Gate: fail if goodput or P95 TPOT regresses by more than `max(5%, 3σ of baseline noise)`.
- Where it runs: hosted CI has no GPU, so it runs on your laptop or as a self-hosted runner.
- Updating the baseline: baseline JSON is committed and only changes in a commit that explains why.

16 Dashboard

Grafana is for watching live stress runs. Numbers in the report come from the harness, not from dashboard screenshots.

Row	Panels
1 — SLO are users getting their SLO?	- TTFT P50/P95/P99, with SLO line - TPOT P95 and ITL P99, with SLO line - Rolling goodput % - Offered, completed and rejected req/s, by reason
2 — Engine why?	- Waiting, running and preempted sequences - Step time P50/P99 - Batch composition per step, stacked: decode sequences vs. prefill tokens - Output tok/s - Queue time P95
3 — Memory	- KV blocks, stacked to total: used, evictable, free - Token-weighted prefix hit ratio, and evictions/s - Preemptions/s - Allocated vs. reserved memory, and allocator retries
4 — Hardware	- SM clock and power: your throttle detector - Temperature - NVML utilization, labelled "kernel-active time, not efficiency" - VRAM

Symptom → where to look

Symptom	Look at
TTFT up, TPOT fine	Queue depth and queue time
TPOT or ITL spikes	Prefill tokens per step and preemptions
Throughput dropped	SM clock and step time
Rejections up	KV used and waiting queue

the offline complement

A per-request Gantt chart built from the step trace, showing queue → prefill chunks → decode → finish. It is the single most useful debugging view you will build.

17 README structure

1. What this is in three sentences, and what it is not.
2. Results at a glance. One table, with a line stating hardware, model, workload, SLO and number of trials.
3. The headline plot: latency vs. offered load for every version and both references.
4. Hardware and constraints: the 6 GB memory math.
5. Architecture: the diagram, and six design decisions with links to their ADRs.
6. The performance story. One short section per version: the bottleneck with profile evidence; hypothesis; change; result; cost; what it revealed next.
7. Performance model: predicted vs. measured, and where it fails.
8. Correctness: what is verified, and how.
9. Failure behavior: the table.
10. Built vs. used.
11. Reproduce: exact commands, expected runtimes, environment manifest.
12. Limitations and next steps.
13. References.

The results-at-a-glance table (to fill with measurements)

Version	Goodput @ SLO	P95 TTFT @ λ	P95 TPOT @ λ	Out tok/s @ λ	Max conc.	Weights / KV VRAM	MBU
V0							
V1							
V2							
V3							
V4							
V5							
V6 / V7							
vLLM							
llama.cpp							

hardware: _____ · model: _____ · workload: _____ · SLO: _____ · trials: _____

screenshots are not results

The README emphasizes engineering decisions and measured results. A Grafana screenshot is context, never evidence.

18 Resume bullets

Every [X] is a number you measure. Never fill one in with a guess.

bullet 1 — the engine

Built a single-GPU LLM inference engine in PyTorch (Qwen2.5-1.5B, RTX 4050 6 GB) with a paged KV-cache block manager, iteration-level continuous batching and chunked prefill. Raised SLO-compliant goodput from [X] to [Y] req/s over a serialized HF baseline on replayed open-loop traces, reaching [Z]% of vLLM on identical hardware and workload.

bullet 2 — memory & reliability

Implemented hash-chained prefix caching with ref-counted LRU eviction, plus memory- and deadline-aware admission control with preemption. Cut P95 TTFT by [X]% at [Y]% shared-prefix traffic, and held goodput at [Z]% of capacity under 2× overload, with zero KV-block leaks across [N]-hour soak tests with random client disconnects.

bullet 3 — performance model & profiling

Built a first-principles step-latency model (bytes moved ÷ measured DRAM bandwidth + fitted overhead) that predicts decode step time within [X]% across batch 1-[B] and context 128-[C]. Used Nsight Systems/Compute to attribute [host launch overhead / pageable H2D copies / full-sequence logits], and removed [X] ms/step with bucketed CUDA-graph capture.

19 LinkedIn project description

Ridgepoint — a single-GPU LLM inference engine (PyTorch, RTX 4050 6 GB). I built an LLM serving engine from the scheduler down to study what limits inference on memory-constrained hardware: a paged KV cache with a ref-counted block allocator, continuous batching with chunked prefill, hash-chained prefix caching, and admission control driven by a step-latency model fitted to measured memory bandwidth. Each version starts from a profiled bottleneck and is benchmarked on replayed open-loop workloads against a naive baseline and vLLM on the same hardware. Result: [goodput X → Y req/s at P95 TTFT ≤ 1 s and TPOT ≤ 75 ms; 2% of vLLM]. The repo includes the engineering report, correctness and failure-injection tests, and the scripts to reproduce every number.

20 Interview questions about this exact project

The grey pointer after each question says where in this notebook the answer lives. Answer from your own measurements, not from the page.

1. Walk through one engine step with 24 decoding sequences and one 512-token prefill chunk. Which tensors are built, what crosses PCIe, and where is the host sync? → §5 *step()*, §6 *T3-T5*, *V5*
2. Your decode step time barely moved from batch 1 to 32. Derive why, and predict the batch size at which it starts rising on your GPU. → §7.2, *E2*
3. Batching amortizes weight reads but not attention. Why? And why is a full-pool step time roughly occupied VRAM ÷ bandwidth regardless of GQA ratio? → §7.2, *E3*
4. Why 16-token blocks? What happens at 1 and at 256, for fragmentation, prefix-sharing granularity and kernel efficiency? → *V3 cost*, *E6*
5. How does your prefix cache decide two blocks are identical? Why chain the parent hash? What happens on a collision? → *V4*, *hash-chain sketch*
6. A request's prompt is 100% cached. How do you produce its first token? → §6 *T3*, *V4 edge cases*
7. Why is `is_causal=True` wrong for chunked prefill in SDPA, and which test catches it?
→ §6 *T4*, §15 *mask test*
8. Recompute vs. swap preemption on your hardware: which wins, by how much, and would the answer flip on an H100 with an MHA 7B model? → *E11*, *App. B*
9. P99 ITL spiked in a load test. Walk from the dashboard to the root cause.
→ §16 *symptom table*, §14 *Nsight*
10. How does admission decide whether a new request can meet its TTFT? What happens when the cost model under-predicts? → §6 *T1/T3*, *V3*, *V5*, *E15*
11. Goodput rose from V2 to V3. Did rejections rise too? How do you report that honestly?
→ §11 *metrics*, §22
12. `nvidia-smi` showed 95% utilization during batch-1 decode. Was the GPU busy? What would you measure instead? → §14 *NVML*, *MBU*
13. Where are the sync points in your step? Which are unavoidable, and how would you hide the last one?
→ §7.8, *V6 stretch*
14. What do CUDA graphs require of your inputs? Why buckets? What breaks if a block table outgrows the captured width? → *V6*, *failure #13*
15. A client disconnects after the scheduler selected its sequence but before outputs were emitted. What happens to its blocks? → §6 *abort path*, *failure #6*
16. Why open-loop arrivals? What is coordinated omission, and how does a closed-loop benchmark hide overload? → §11 *replayer*

17. Your first INT8 weight-only attempt was slower than BF16. Why? At what batch size does even the fused version stop helping? → V7, E13
18. You reach X% of vLLM's goodput. Where exactly is the remaining gap, with evidence? → E5, day 26
19. Greedy output at batch 1 diverges from batched output after 150 tokens. Bug or not? How do you decide? → §7.9, E17
20. Scale this to 8 GPUs. Tensor parallel or replicas? What is communicated per layer, why is TP over PCIe painful, and what does having only 2 KV heads constrain? → §23

21 What you must be able to answer before this goes on your resume

Answer each out loud, without notes, in under two minutes. Tick it only when you can.

- ☐ From `config.json` alone: KV bytes per token, KV pool size on your card, and the maximum number of concurrent 1k-token sequences.
- ☐ The batch-1 decode ceiling and prefill floor from your own E1 measurements, and the gap between them and what you measured.
- ☐ Each metric in your tables exactly as your code computes it: TTFT start and end points, ITL pooling, TPOT denominator, the goodput condition, and why hit rate is token-weighted.
- ☐ Every README plot: axes, workload, SLO, what the knee means, and why each version's curve moved.
- ☐ The block allocator and the prefix-cache insert/lookup/evict logic, written on a whiteboard in 20 minutes, including every refcount transition.
- ☐ The attention computation for a prefill chunk over cached KV, including mask alignment and positions.
- ☐ Every default value — block size, token budget, `max_num_seqs`, watermark, graph buckets, SLOs — each backed by the experiment that chose it.
- ☐ For every failure in your table: what happens step by step, and how the test proves it.
- ☐ One real bug that a test or profile caught, and how you would have missed it otherwise.
- ☐ Where your performance model fails, and why.
- ☐ The biggest gap to vLLM, explained with profiles rather than guesses.
- ☐ Which of your results are within run-to-run noise.
- ☐ What vLLM or SGLang does differently for each component you built, and why they chose it.
- ☐ What changes on an H100: which bottlenecks disappear and which get worse. (For example, host overhead matters more there, because GPU steps are about 15× faster.)
- ☐ A headline number reproduced from a clean clone.
- ☐ Every line of code, including anything an AI assistant helped write.

the bar

If any answer is "the library handles that" or "I think", you are not ready. Go run the experiment.

22 Anti-patterns

Things that do not belong in the project

- ✗ A chat UI, LangChain, or a RAG demo as the front page.
- ✗ Redis, Kafka, Kubernetes or microservices for one GPU.

Measurement mistakes

- ✗ Numbers without hardware, model, workload, SLO and number of trials.
- ✗ Means instead of percentiles.
- ✗ P99 from 200 requests.
- ✗ Closed-loop-only load.
- ✗ No warm-up exclusion.
- ✗ Single runs.
- ✗ Output length left to EOS in benchmarks, which makes it an uncontrolled variable.
- ✗ "Tokens/sec" without saying input, output or total.
- ✗ Throughput reported without latency.

Misleading comparisons and claims

- ✗ Comparing only against your own strawman baseline, with no vLLM or llama.cpp reference.
- ✗ GPU utilization as the efficiency headline.
- ✗ Goodput gains that quietly come from rejecting more requests.
- ✗ Optimizations with no profile motivating them and no stated cost.
- ✗ Buzzword README: "production-ready", "blazing fast", or any claim the evidence does not support.
- ✗ Hiding negative results. They are part of your credibility.

Engineering hygiene

- ✗ A fast engine with no equivalence tests.
- ✗ Copying vLLM's directory layout, class names or kernels, or forking a nano-vLLM-style repo. Reviewers recognize these immediately.
- ✗ Grafana screenshots presented as results.
- ✗ One giant commit, or versions as copy-pasted folders.
- ✗ Magic constants with no experiment behind them.
- ✗ AI-generated code you cannot explain. Interviewers probe exactly the parts you did not write.

23 Production evolution

the principle behind the design

Scheduling within a replica runs on a millisecond cadence against exact local state; routing across replicas runs on a seconds cadence against approximate global state. That is why Ridgepoint's scheduler is per-replica: it is the building block, not the whole system.

1 GPU – the production-grade version of what you built

The architecture stays the same. What you would add:

- Multi-LoRA batching.
- Speculative decoding. It trades spare compute for fewer memory-bound steps. Ideal on bandwidth-poor hardware, but it helps less at high batch sizes.
- Structured output.
- Better kernels, such as FlashInfer or FlashAttention-3 on Hopper.
- FP8 weights and activations.
- Overlapped CPU/GPU scheduling.
- Tokenization in separate processes.

8 GPUs – one node

► Replicas vs. tensor parallelism (TP)

- If the model and its KV fit on one GPU, 8 data-parallel replicas usually win on throughput, with zero communication.
- TP is for models or KV that do not fit, or to cut per-token latency.

► How Megatron-style TP works

- Column-parallel QKV and gate/up projections; row-parallel o_proj and down projections.
- That means **two all-reduces of [tokens × hidden] per layer**.
- Decode all-reduces are small and latency-bound, so NVLink matters. Over PCIe, TP often loses.

► Constraints

- KV heads shard across TP ranks, so TP degree should not exceed the number of KV heads. Qwen2.5-1.5B has 2, so TP=4 would force replicating KV heads.
- One scheduler drives each TP group in lockstep, and CUDA graphs must capture the NCCL collectives.
- MoE models add expert parallelism with all-to-all communication.

► Routing

A router in front of the replicas combines least-outstanding-tokens with prefix affinity.

100 GPUs

► Parallelism and disaggregation

- Pipeline parallelism is only for models larger than a node. You pay for pipeline bubbles and need micro-batching.
- Disaggregated prefill and decode pools (DistServe, Splitwise):
 - prefill is compute-bound and decode is bandwidth-bound, so size separate pools to the workload's input/output ratio;
 - KV transfers over RDMA/NVLink;
 - you trade transfer cost and complexity for eliminating interference completely.

► KV tiering

GPU → CPU → SSD or a remote store, for multi-turn reuse.

► Cache-aware routing

- Consistent hashing of prefix-block hashes to replicas, with bounded load.
- This creates a real tension: **cache locality vs. load balance**.
- The router keeps approximate per-replica cache state.

► Autoscaling

- Scale on queue time, KV utilization and goodput, not CPU.
- Model cold start (tens of GB of weights) drives the design: warm pools, and weight streaming from NVMe or peer GPUs.

► Failure domains

- GPU hardware errors, and replica draining.
- Streamed requests cannot be retried transparently after tokens have been sent.

Systems such as NVIDIA Dynamo, llm-d and SGLang's router explore this space.

Thousands of requests / s

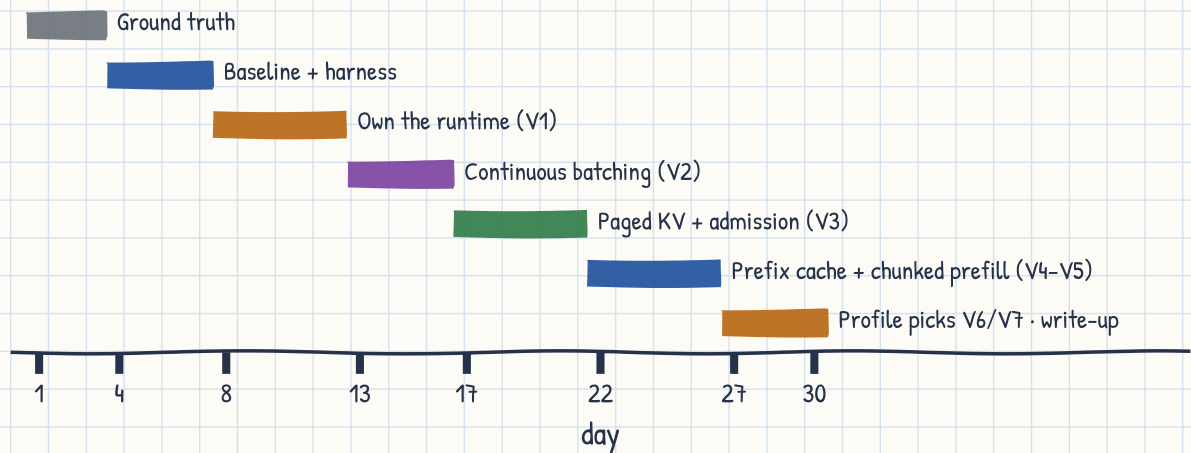
- **Frontends**: stateless and horizontally scaled, with tokenization offloaded, and gRPC between router and engines.
- **Traffic control**: per-tenant quotas and rate limits; priority classes, where batch traffic fills idle capacity and is preempted first; global admission based on predicted capacity.
- **Observability**: per-replica SLO attribution and tail-latency analysis.
- **Cost**: cost per million tokens becomes a first-class metric.

a local proof

The stretch `sim/` component is a discrete-event simulator of N replicas driven by your validated cost model. With it you can compare round-robin, least-loaded and prefix-affinity routing using evidence rather than prose.

24 30-day build plan

Assumes 5–6 focused hours per day. Part-time? Treat it as a 60-day plan: stretch the calendar, never shortcut the measurement.



seven phases — the harness comes before the engine

Days 1–3 — Establish ground truth

- ☐ **Day 1** — environment and microbenchmarks. Native Linux, driver, CUDA and PyTorch, with a pinned uv environment and a Docker image. Confirm the discrete GPU carries no display processes. Read Horace He's "Go Brrrr" post. Write the E1 microbenchmarks.
- ☐ **Day 2** — throttling and memory math. Run E1 plugged in and on battery, learn how your laptop throttles, and write the run-validity rules. Hand-write `memory-budget.md` for every model: KV per token, weights, pool size, and the decode ceiling.
- ☐ **Day 3** — reading and definitions. Read Orca and the scheduling and block-management sections of the vLLM paper. Freeze the SLOs and the six workload definitions.

Days 4–7 — Baseline and harness (the harness is the product)

- ☐ **Day 4** — the V0 server with full instrumentation and vLLM-compatible SSE.

- ☐ **Day 5** – trace generator, open-loop replayer and null backend. Prove the replayer can drive 500 streams with P99 lag under 5 ms.
- ☐ **Day 6** – the analysis pipeline: per-request tables, bootstrap CIs, goodput and the headline plot.
- ☐ **Day 7** – first comparison. Run V0, vLLM and llama.cpp on every workload. Write the first report section. Take one PyTorch profile of V0 to learn the tool.

Days 8–12 – Own the runtime

- ☐ **Days 8–9** – the Qwen2 forward pass from safetensors, a contiguous KV cache, and prefill/decode functions. Compare every layer against HF.
- ☐ **Day 10** – correctness, then the first profiles. Sampler, logits on the last position only, and the correctness suite, including the mask-alignment test. Measure batch-1 TPOT against the ceiling. Take your first Nsight timeline and find the sync points.
- ☐ **Day 11** – V1 static and windowed batching, the step trace, and E2.
- ☐ **Day 12** – E4 part 1, and a first cost-model fit on the batch dimension. Write V1's section, including what surprised you.

Days 13–16 – Continuous batching

- ☐ **Days 13–14** – engine loop, sequence states, per-slot lengths, streaming driven by engine outputs, the abort path, and the `EngineClient` boundary (in-process for now).
- ☐ **Day 15** – E4 part 2, E5 for V2, and the ITL timeline showing prefill spikes.
- ☐ **Day 16** – the concurrency test with random disconnects. **Something will break; fix it and write down what it was.**

Days 17–21 – Paged KV and admission control

- ☐ **Day 17** – BlockAllocator plus property tests, on CPU only, before any GPU code.
- ☐ **Day 18** – pool sizing from a profiling run, `slot_mapping` writes, block tables, and reference paged attention with shuffled-table tests.
- ☐ **Day 19** – admission with a watermark, recompute preemption, and the preemption-equivalence test.
- ☐ **Day 20** – backpressure (429 with `Retry-After`), E10, and E6.
- ☐ **Day 21** – buffer day. Run failure tests 1–9 and write the V3 section.

Days 22–26 – Prefix caching and chunked prefill

- ☐ **Days 22–23** – hash-chain prefix cache, LRU evictable set, the "prefill over cached KV" path, the hit-vs-miss equivalence test, and E8.

- ☐ **Day 24** – chunked prefill on the same primitive, the token budget, and E9.
- ☐ **Day 25** – refit the cost model with context and prefill terms (E15), and wire it into the dynamic budget and admission.
- ☐ **Day 26** – E5 headline curves for V0–V5 plus vLLM. Profile both engines at OP-B and write the gap attribution.

Days 27–30 – Let the profile choose, then write

- ☐ **Days 27–28** – one of V6 or V7, done well. If the V5 profile shows material GPU idle gaps, do V6: decode CUDA graphs and persistent buffers. If steps are bandwidth-saturated, do V7: weight INT8 naive vs. fused, and FP8 KV. Put the other on the next-steps list.
- ☐ **Day 29** – the report and README, including negative results and limitations.
- ☐ **Day 30** – reproducibility. From a fresh clone, run `make setup` && `make bench-smoke`. Commit the regression baseline. Tag each version.

if you fall behind, cut in this order

1. V6 or V7.
2. E11, E12, E14, E16.
3. The Triton kernel.
4. Grafana.

never cut

- ✓ the harness validity rules;
- ✓ correctness tests;
- ✓ the vLLM comparison;
- ✓ the report;
- ✓ failure tests 1, 5 and 6.

After day 30

- ☐ The Triton split-KV decode kernel (E7).
- ☐ The process split and GIL experiment (E12b).
- ☐ Swap preemption (E11).
- ☐ The full quantization matrix (E13).
- ☐ The 8B offload experiment (E14).
- ☐ The multi-replica simulator.

25 Final recruiter test

What makes this meaningfully better than a standard LLM-serving project?

- Standard projects wrap an engine, and their decisions are about the API.
- Here the decisions are about GPU memory, batch composition and time. They are made by code you wrote, measured by a harness that measures vLLM the same way, and explained by a model that predicts them.
- The unit of the project is the experiment, not the endpoint.

Which parts show real engineering judgment?

- ✓ Choosing a 1.5B model because of the memory math, not the largest model that loads.
- ✓ Rejecting Redis and microservices, and splitting processes only when a measurement justifies it.
- ✓ Headlining goodput under SLO, with rejections reported next to it, instead of raw throughput.
- ✓ Open-loop load, validity rules and variance reporting.
- ✓ Letting the profile choose between V6 and V7.
- ✓ Keeping failed hypotheses in the report, such as naive INT8 being slower, or caching reducing concurrency under pressure.
- ✓ Stating what the project does not prove.

What would a strong ML-systems reviewer inspect first?

1. The headline plot with vLLM on it, and whether hardware, workload and SLO are stated.
2. kvcache/ : refcounting, eviction, invariants, property tests.
3. The scheduler's step function: readable, deterministic, testable without a GPU?
4. The numerics tests, especially hit-vs-miss and preemption equivalence.
5. bench/ methodology: open-loop load, warm-up exclusion, percentiles from raw data, trial counts.
6. The "what it cost" line in each report section, and the negative results.
7. The commit history: incremental, with measurements between optimizations.

What could still make it look weak?

- ✗ An unexplained gap to vLLM. A large gap is fine if you attribute it with evidence.
- ✗ Noisy laptop numbers with no variance reported.
- ✗ Breadth without depth. A half-finished V6 or V7 looks worse than one that is absent.
- ✗ Only library attention kernels. That reads as "scheduler person, not GPU person"; the Triton kernel fixes it.
- ✗ Toy workloads with fixed lengths only.
- ✗ Code you cannot explain line by line.

- ✗ Absolute numbers that do not transfer. A small model on a laptop is a limitation. Frame results as mechanisms and ratios: percentage of bandwidth ceiling, percentage of vLLM on the same hardware.

Which single feature adds the most technical credibility?

the validated performance model

$$\text{step time} \approx \alpha + \text{bytes} \div BW_{\text{eff}} + \text{prefill FLOPs} \div F_{\text{eff}}$$

three constants, fitted from your step trace

- How it is validated: on held-out operating points, with a predicted-vs-measured plot and an honest account of where it breaks. Small batches expose host overhead; long contexts expose attention-kernel inefficiency.
- How it is used: it drives the scheduler's token budget and admission decisions against the TPOT SLO.
- Why it beats a custom kernel: it shows you understand the whole system, not one operator; it makes every other result interpretable; it turns "measurable" into "controllable"; and it almost never appears in portfolios.

Runner-up: a Triton split-KV paged decode kernel, reported as a percentage of measured peak bandwidth.

A Formula cheat sheet

Every equation in this notebook on two pages. N = parameters · W = weight bytes · BW = bandwidth · B = batch · P = prompt tokens.

Quantity	Formula
Weight bytes	$W = N \times \text{bytes per parameter (BF16 = 2)}$
KV bytes per token	$2 \times L \times H_{kv} \times D \times \text{bytes per element}$
KV pool tokens	$\text{pool bytes} \div \text{KV bytes per token}$
Blocks per sequence	$\lceil \text{context} \div \text{block_size} \rceil$
Max concurrent sequences	$\lceil \text{num_blocks} \times (1 - \text{watermark}) \div \text{blocks per sequence} \rceil$
Contiguous-reservation waste	$\sum (\text{max_len} - \text{actual}) \text{ tokens}$
Paged waste	$< 1 \text{ block per sequence}$
Arithmetic intensity	$\text{FLOPs} \div \text{bytes moved}$
Ridge point	$\text{peak FLOP/s} \div \text{peak bandwidth}$
Decode AI, weights	$\approx B$
Decode AI, attention	$\approx \text{query heads} \div \text{KV heads (GQA group)}$
Batch-1 decode ceiling	$t \geq W \div BW$
Decode step time model	$t \approx \alpha + (W + \sum_i \text{KV}(\text{ctx}_i)) \div BW$
Full-pool decode step	$\approx (W + \text{pool bytes}) \div BW$
Prefill FLOPs	$\approx 2 \times N \times P + \text{quadratic attention term}$
Prefill floor	$t \geq 2 \times N \times P \div \text{achieved FLOP/s}$
Full step-time model (E15)	$t \approx \alpha + \text{bytes} \div BW_{\text{eff}} + \text{prefill FLOPs} \div F_{\text{eff}}$
MBU	$\text{modeled bytes per decode step} \div \text{GPU step time} \div \text{measured peak BW}$
Logits memory	$\text{rows} \times \text{vocab} \times 4 \text{ B (FP32)}$
TTFT	$\text{queue} + \text{tokenize} + \text{prefill steps} + \text{first sample} + \text{transport}$
TPOT	$(E2E - \text{TTFT}) \div (n_{\text{out}} - 1)$
E2E	$\approx \text{TTFT} + (n_{\text{out}} - 1) \times \text{TPOT}$
Goodput	$\text{req/s meeting P95 TTFT and P95 TPOT SLOs}$
Little's law	$L = \lambda \times W_{\text{latency}}$
M/M/1 queue wait	$\propto 1 \div (1 - \rho)$, with $\rho = \lambda \div \mu$
Swap preemption	$2 \times \text{KV}(\text{ctx}) \div \text{PCIe bandwidth}$

Quantity	Formula
Recompute preemption	$\approx 2 \times N \times \text{ctx} \div \text{achieved FLOP/s (minus prefix-cache hits)}$
Two-tier offload (E14)	$\dagger \approx \text{GPU bytes} \div \text{GPU BW} + \text{CPU bytes} \div \text{host BW}$
Prefix hit ratio	$\text{cached prompt tokens} \div \text{prompt tokens (token-weighted)}$
Regression gate	fail if regression > max(5%, 3σ of baseline noise)
Block hash	$h_i = H(h_{i-1}, \text{tokens of block } i), h_{-1} = H(\text{model id})$

B Worked numbers from config.json

Qwen2.5-1.5B-Instruct, worked end to end. Redo this page by hand on day 2, then again with your EI numbers.

1. Read the config

config.json — the fields that matter

hidden_size	1536	
num_hidden_layers	28	
num_attention_heads	12	# head_dim = 1536 / 12 = 128
num_key_value_heads	2	# GQA group = 12 / 2 = 6
intermediate_size	8960	
vocab_size	151936	
tie_word_embeddings	true	# lm_head reuses the embedding matrix

2. Parameters and weight bytes

per layer: $q (1536^2 + \text{bias}) + k, v (1536 \times 256 + \text{bias each}) + o (1536^2) +$
MLP ($3 \times 1536 \times 8960$) + 2 norms $\approx 46.8 \text{ M}$

28 layers $\approx 1.310 \text{ B}$ + embedding $151,936 \times 1536 \approx 0.233 \text{ B}$

$\Rightarrow N \approx 1.54 \text{ B}$

$W = 1.54 \text{ B} \times 2 \text{ bytes} \approx 3.09 \text{ GB}$

tied embeddings: decode still streams the 467 MB embedding matrix once per step, as the
lm_head

3. KV cache

KV / token = $2 \times 28 \times 2 \times 128 \times 2 \text{ B} = 28,672 \text{ B} = 28 \text{ KiB}$

pool $\rightarrow$ tokens (16-token blocks): 1.5 GB $\rightarrow$ 52.3k (3,270) · 1.8 GB $\rightarrow$ 62.8k (3,924) · 2.0 GB $\rightarrow$ 69.8k (4,360)

Question	Working (1.8 GB pool)	Answer
Concurrent 1k-token sequences, paged	$1024 \div 16 = 64$ blocks each; $3,924 \div 64$	≈ 61
Contiguous reservation at $S_{\max} = 40\%$	$62,779 \div 40\%$	15
Paged, mean actual length 600	$\lceil 600/16 \rceil = 38$ blocks; $3,924 \div 38$	≈ 103
Same pool, Qwen3-1.7B (112 KiB/token, ~ 1.5 GB pool)	$1.5 \text{ GB} \div 114,688 \text{ B} \approx 13.1\text{k}$ tokens	$\approx 12 \times 1\text{k}$ seqs

Subtract the watermark (1–5%) before quoting a concurrency number.

4. Decode bounds **ESTIMATE**

Case	Bytes per step	Floor at 192 GB/s	Output tok/s bound
Batch 1, short context	$\approx 3.09 \text{ GB}$	16.1 ms	≈ 62
Batch 32, 1k context each	$3.09 \text{ GB} + 32 \times 1024 \times 28 \text{ KiB} (0.94 \text{ GB}) = 4.03 \text{ GB}$	21.0 ms	$\approx 1,525$
Full 1.8 GB pool	$3.09 + 1.8 \approx 4.9 \text{ GB}$	$\approx 25 \text{ ms}$	depends on how many sequences share the pool

5. Prefill and memory traps

Case	Working	Answer
Prefill 2,048 tokens	$2 \times 1.54 \text{ B} \times 2048 \approx 6.3 \text{ TFLOP}$	$0.42 \text{ s} @ 15 \text{ TFLOPS} \cdot 0.25 \text{ s} @ 25$
Logits for every prompt position	$2048 \times 151,936 \times 4 \text{ B}$	1.24 GB
Logits, last position, 64 sequences	$64 \times 151,936 \times 4 \text{ B}$	39 MB

6. Preemption: swap vs. recompute (E11)

Case	Swap (out + in)	Recompute	Verdict
Qwen2.5-1.5B, 2k ctx, laptop	$2048 \times 28 \text{ KiB} = 58.7 \text{ MB}$ (56 MiB) each way; $\approx 5 \text{ ms}$ each at $\sim 12 \text{ GB/s}$ PCIe (measure in E1)	$\approx 6.3 \text{ TFLOP} \approx 0.42 \text{ s}$	swap wins $\approx 25\text{--}40\times$
MHA 7B, 2k ctx, H100	$512 \text{ KiB/token} \rightarrow 1.07 \text{ GB}$ each way; $\approx 21 \text{ ms}$ each at $\sim 50 \text{ GB/s}$	$\approx 27 \text{ TFLOP} \approx 46 \text{ ms}$ at $\sim 600 \text{ TFLOPS}$ achieved	close to even

Why it flips: GQA shrinks KV bytes $\times 16$ on the small model, while the laptop's compute is $\sim 40\times$ below an H100's. Bytes got cheaper, FLOPs stayed expensive.

Glossary

Admission control

Deciding whether a request may enter the waiting queue or the running batch, based on memory and predicted latency.

α (alpha)

The fixed per-step overhead in the step-time model: launches, host work, sampling.

Arithmetic intensity

FLOPs performed per byte moved from memory.

Backpressure

Pushing overload back to clients (429 + Retry-After) instead of queuing without bound.

Block / block table

A fixed-size chunk of KV (16 tokens here); the per-sequence list mapping logical to physical blocks.

Chunked prefill

Splitting a prompt's prefill across steps so each step stays within a token budget.

Continuous batching

Batch membership changes every step; finished sequences leave and new ones join immediately.

Coordinated omission

A closed-loop client sends less when the server slows down, hiding the latency real users would see.

CUDA graph

A recorded kernel sequence with fixed shapes and addresses, replayed with one launch.

Decode

Generating one token per sequence per step; bandwidth-bound.

Disaggregation

Running prefill and decode on separate GPU pools and moving KV between them.

E2E latency

From send to the last token.

Evictable set

Cached blocks with refcount 0, kept in LRU order until the allocator needs them.

GQA

Grouped-query attention: several query heads share one KV head, shrinking the KV cache.

Goodput

Requests per second that meet all SLOs.

HOL blocking

Head-of-line blocking: work behind a slow item waits even though it could run.

ITL

Inter-token latency: the gap between consecutive tokens, pooled across tokens.

KV cache

Stored keys and values for every past position, layer and KV head.

MBU

Model bandwidth utilization: modeled bytes per step $\div$ step time $\div$ peak bandwidth.

NVTX

Named ranges you add in code so Nsight timelines show engine phases.

Open-loop load

Requests sent on a fixed schedule regardless of completions.

Paged attention

Attention that reads KV through a block table from non-contiguous memory.

Pinned memory

Page-locked host memory; required for truly asynchronous H2D copies.

Preemption

Evicting a running sequence's KV (recompute or swap) to let others continue.

Prefill

Processing the prompt in parallel to fill the KV cache; compute-bound.

Prefix caching

Reusing KV blocks of identical prompt prefixes across requests.

Ridge point

The arithmetic intensity where a kernel stops being memory-bound.

Roofline

$\min(\text{peak FLOP/s}, \text{AI} \times \text{bandwidth})$: the performance ceiling for a given AI.

slot_mapping

For each new token, the physical slot where its K and V are written.

SLO

Service-level objective, e.g. $\text{P95 TTFT} \leq 1 \text{ s}$.

Split-KV (Flash-decoding)

Parallelizing decode attention along the sequence and merging with log-sum-exp.

Sticky CUDA error

An error (e.g. illegal memory access) that poisons the context until process restart.

Token budget

The maximum tokens (decodes + prefill chunks) processed in one step.

TPOT

Time per output token after the first, averaged per request.

TTFT

Time to first token, as the client observes it.

Watermark

Blocks held in reserve so running decodes can keep growing.

Experiment log

Copy this page for every experiment. The shape matches the `experiments/Exx/README.md` template.

Experiment
Git SHA · config hash

Date

Trials

Question

Hypothesis — with a predicted number, from §7 or the performance model

Setup — hardware, model, workload, config, cache state (cold / warm)

Result — table or plot, with confidence intervals

Interpretation · Cost · Next bottleneck

Validity check

☐ generator lag $P99 \leq 5$ ms

☐ no throttling (SM clock)

☐ no unintended errors

☐ enough requests in window

☐ Little's law within 10%

☐ ≥ 3 trials, spread reported

Profile findings

One paragraph per profile — this becomes `profiling/FINDINGS.md`.

Version	OP-A · batch 1, 512 ctx	OP-B · 32 seqs, ~1k ctx	OP-C · mixed @ 70%
V0			
V1			
V2			
V3			
V4			
V5			
V6 / V7			
vLLM (OP-B)			

Bug log

Interview question 21.9 asks for one real bug a test or profile caught. Record them as they happen.

Day	Symptom	Caught by	Root cause · fix